3.4. Управление памятью Windows и macOS глубже

Working Set · Commit · сжатие · файл подкачки · своп · Memory Pressure

Windows

Резерв → Commit → Working Set

давление памяти

сокращение → сжатие → файл подкачки

macOS

страницы в RAM → сжатие → своп

Go

Достижимость ≠ активность

Две ОС используют разные API и метрики, но решают одну физическую задачу: удержать горячий рабочий набор в RAM и вытеснить менее ценные данные с минимальной ценой.

Одна физическая проблема - разные языки ОС

виртуальный адрес → физические страницы → рабочий набор → давление памяти → вытеснить или сжать → файл подкачки и своп → исключение → задержка

Windows

Working Set · Commit · Transition/Standby · сжатие · файл подкачки

macOS

страницы RAM · wired · сжатие · своп · Memory Pressure

Сравниваем причинные цепочки от резидентности страниц до задержки.

Что у всех ОС одинаково

Виртуальный адрес

Адрес есть в виртуальном пространстве процесса. Это еще не гарантирует наличие физической страницы в RAM.

Физическая RAM

Ядро решает, какая часть виртуальной памяти прямо сейчас реально находится в RAM.

Виртуальная аллокация и резидентность - разные состояния одной памяти.

Что меняется между ОС

- терминология и учет памяти;
- правила вытеснения и освобождения страниц;
- сжатие и хранение содержимого;
- работа файла подкачки и свопа;
- инструменты диагностики;
- ограничения ресурсов и контейнерные механизмы;
- пользовательские сводные метрики.

Главный вопрос ядра везде один: какие данные должны остаться в дорогой быстрой RAM прямо сейчас?

Главная сравнительная карта

Общая физика

виртуальный адрес
↓
физические страницы
↓
резидентный набор
↓
давление памяти

Windows

сокращение Working Set
Transition / Standby
сжатие
файл подкачки

macOS

резидентные / закрепленные
сжатие
своп
Memory Pressure

Дальше все детали будем привязывать к этой карте.

Windows: три уровня, которые нельзя смешивать

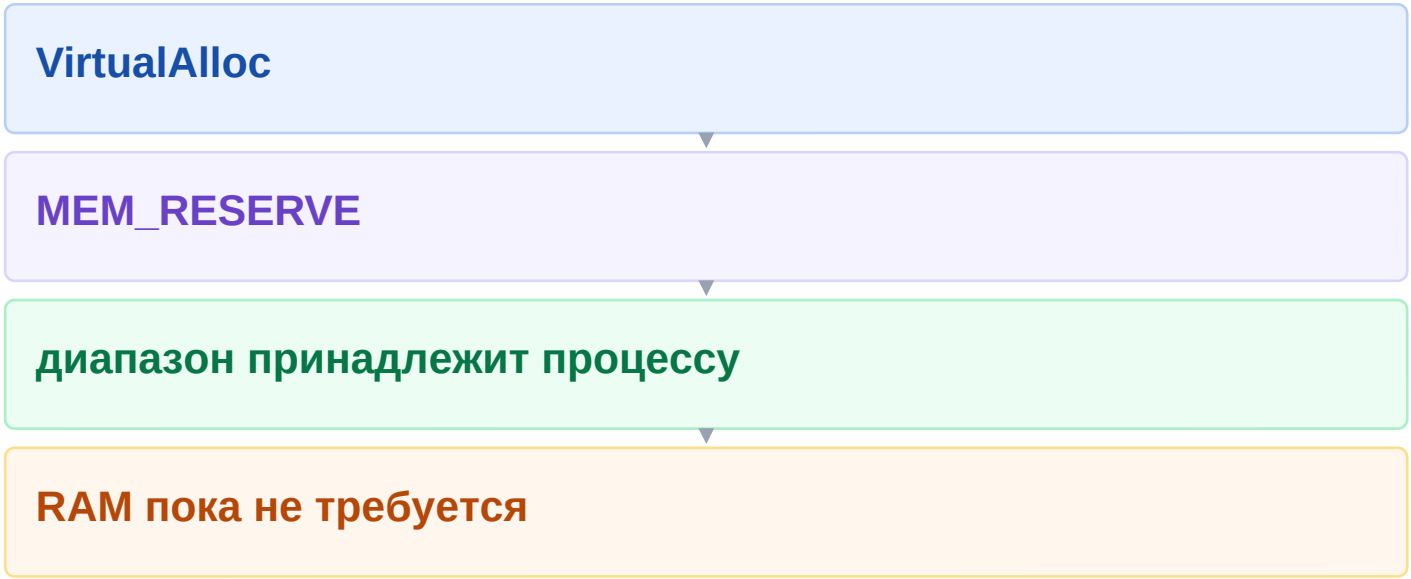
Виртуальное адресное пространство
какие адреса принадлежат процессу

Commit
объем обеспеченной системой памяти

Working Set
какие страницы находятся сейчас в RAM

Reserve, Commit и Resident отвечают на три разных вопроса.

Виртуальное пространство: сначала резервируем адреса



Reserve закрепляет виртуальные адреса, но не занимает RAM.

MEM_RESERVE не резервирует физическую память

Есть

Адресный диапазон закреплен за процессом; другие отображения туда не попадут.

Еще нет

Физическая RAM и место в файле подкачки пока не обязаны существовать.

Большой резерв адресов почти ничего не говорит о расходе RAM.

MEM_COMMIT: система гарантирует обеспечение памяти



Commit означает больше, чем резерв, но не равен резидентности.

Commit и Resident - разные состояния

Учтенная память

Windows гарантирует обеспечение содержимого страниц.

В RAM

Конкретная физическая страница сейчас находится в RAM и входит в Working Set.

Страница с Commit может находиться вне RAM.

Commit Charge: сколько памяти система обеспечила

растет объем приватной памяти с Commit

▼

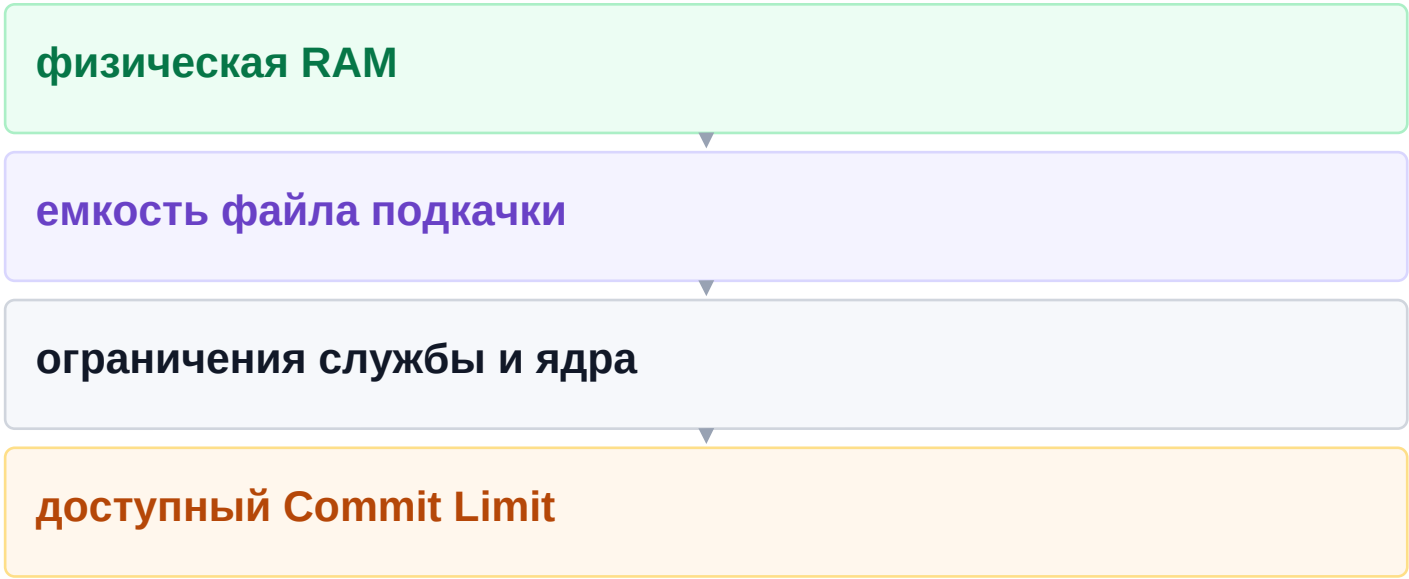
общий Commit Charge ↑

▼

сравнить с Commit Limit

Commit отражает обязательства системы, а не занятую RAM.

Commit Limit: предел обеспечения памяти



Файл подкачки влияет и на подкачку, и на общий лимит Commit.

Почему выделение памяти не удается при свободной RAM

Что видит инженер

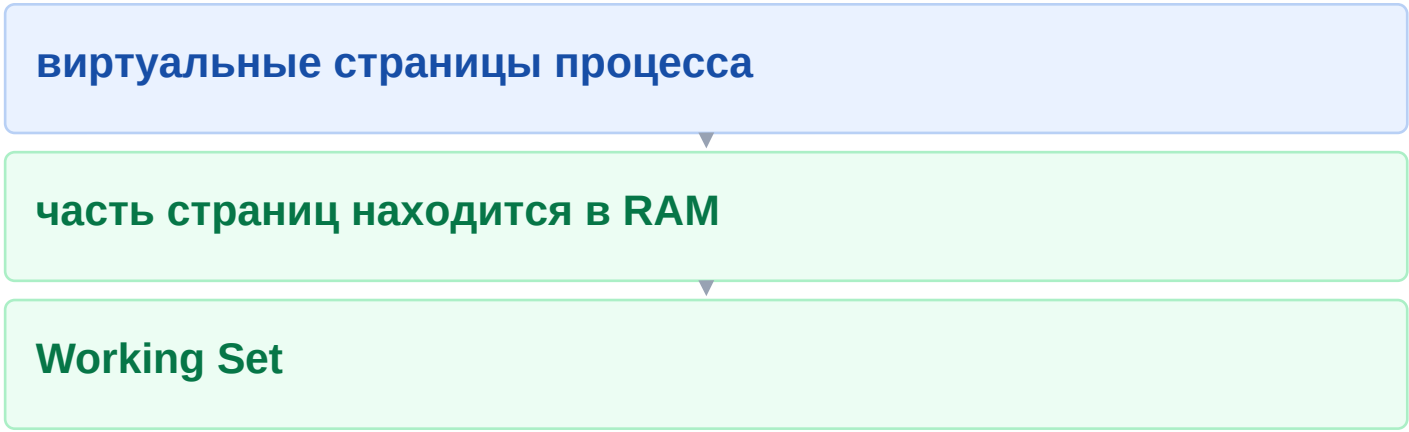
В системе еще есть свободные физические страницы.

Что видит диспетчер памяти

Commit Charge подошел к Commit Limit: новое обещание обеспечить содержимое памяти уже нельзя.

Исчерпание Commit и нехватка RAM — разные проблемы.

Working Set — резидентные страницы процесса



Working Set отвечает на вопрос: какие страницы процесса прямо сейчас находятся в RAM?

Commit и Working Set - независимые оси

Процесс A

Commit = 4 GiB
Working Set = 800 MiB

Большой Commit, но мало страниц в RAM.

Процесс B

Commit = 1 GiB
Working Set = 900 MiB

Почти вся учтенная память сейчас находится в RAM.

Одинаковый Commit не определяет Working Set, и наоборот.

Working Set постоянно меняется

страничное исключение обработано

↓

страница появилась в RAM

↓

включается в Working Set

↓

давление памяти

↓

сокращение Working Set

↓

страница покидает Working Set

Working Set меняется вместе с давлением на память.

Мягкое исключение: страницы нет в Working Set, но она есть в RAM



Мягкое исключение требует работы VM, но не чтения с диска.

Жесткое страничное исключение: нужны данные с диска



Жесткое исключение добавляет задержку накопителя вместо DRAM.

Page Faults/sec легко понять неправильно

Высокий счетчик

Может состоять в основном из мягких исключений без дискового I/O.

Что важно при расследовании

Отделить жесткие исключения / чтение страниц и сопоставить с задержкой.

Число исключений не равно числу операций ввода-вывода.

Сокращение Working Set освобождает RAM, но не владение памятью

RAM нужна другим процессам

↓

диспетчер сокращает Working Set

↓

страница больше не в рабочем наборе

↓

виртуальный адрес сохраняется

↓

Commit может сохраниться

Сокращение Working Set не обязательно меняет адреса или Commit.

После сокращения Working Set страница может остаться в RAM



Промежуточные состояния позволяют вернуть страницу без чтения с диска.

Зачем нужны промежуточные состояния страниц

Без них

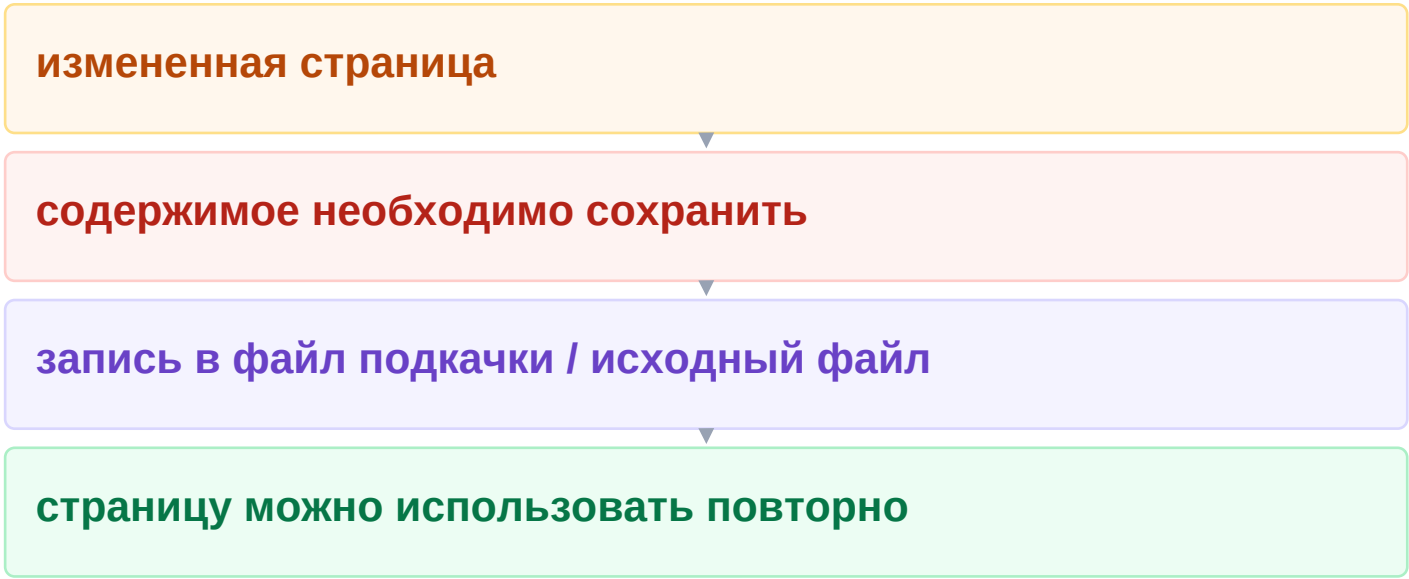
Кратковременное давление сразу превращало бы вытеснение в массовые обращения к диску.

С ними

Недавно исключенная страница может еще находиться в RAM и быстро возвращаться в рабочий набор.

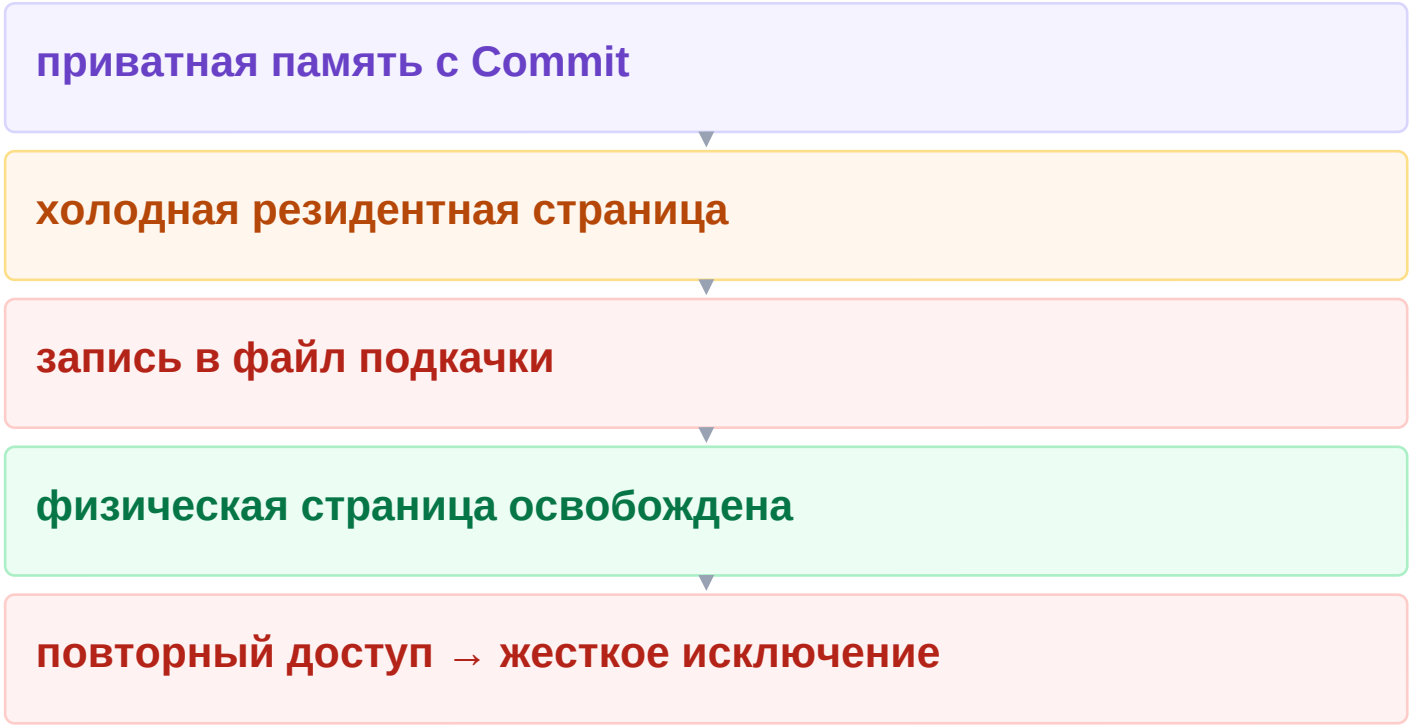
Между Working Set и диском существует дешевая промежуточная ступень.

Измененную страницу нельзя сразу отдать другим



Измененные страницы требуют записи перед освобождением.

Файл подкачки: обеспечение приватной памяти с Commit



Файл подкачки освобождает RAM, но повторный доступ обходится дороже.

Файл подкачки — не просто «медленная RAM»

Роль в Commit

Увеличивает возможности обеспечения памяти.

Роль в подкачке

Хранит приватные данные, когда физическая страница нужна другим задачам.

Файл подкачки влияет и на учет Commit, и на подкачку.

Файл подкачки полезен и при большом объеме RAM

Что дает

Больше запаса Commit и возможностей вытеснить действительно холодные приватные страницы.

Чего не означает

Наличие файла подкачки еще не означает проблему. Нужна реальная активность подкачки.

Измеряем активность подкачки и жесткие исключения, а не наличие файла.

Memory Priority: подсказка ядру о ценности страниц

Важные данные

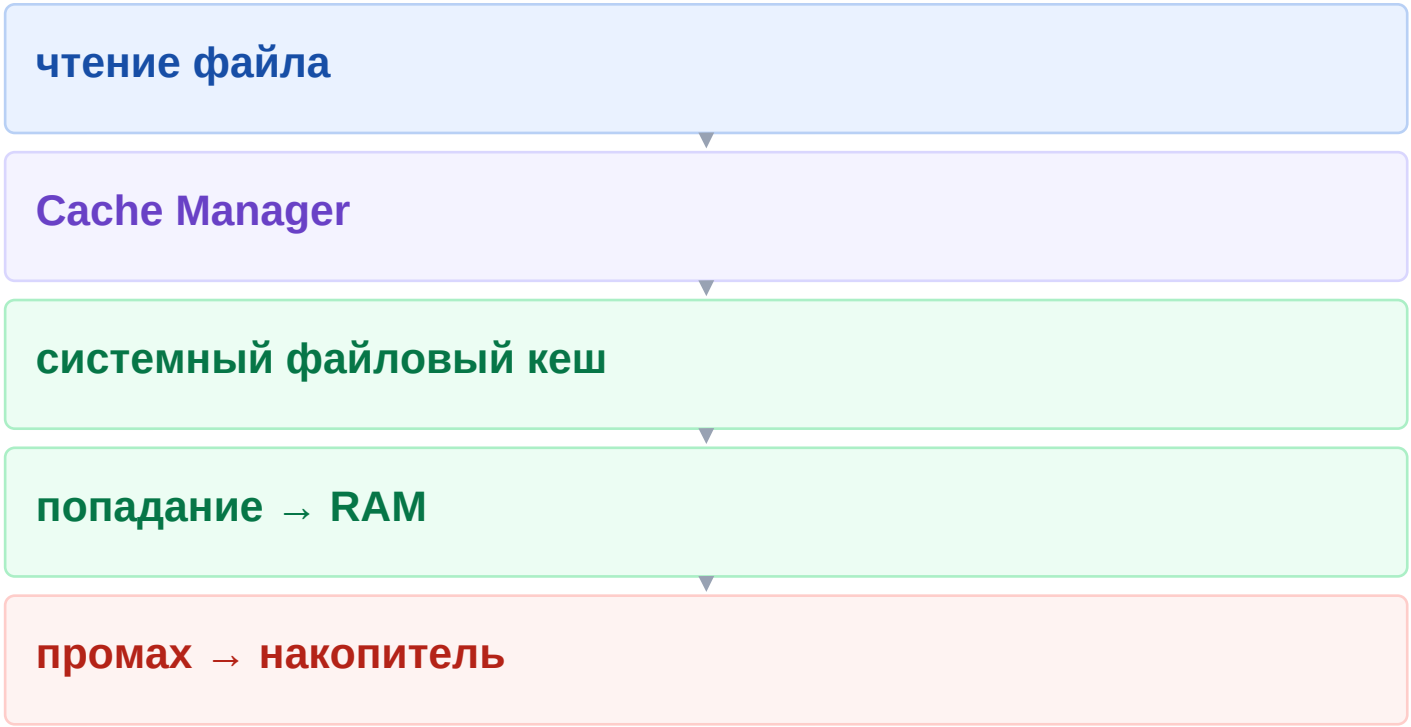
Активный кеш запросов желательно дольше держать в RAM дольше.

Менее важные данные

Фоновые проходы по данным можно при прочих равных вытеснить раньше.

Приоритет — рекомендация диспетчеру памяти, не гарантия.

System File Cache: RAM используют и файловые данные



Большой файловый кеш может быть нормальным использованием RAM.

Большой кеш еще не означает утечку

Нормально

Кеш ускоряет чтение и может освобождаться при давлении памяти.

Проблема

Кеш вытесняет более полезные страницы, растут исключения и операции ввода-вывода.

Смотрим последствия давления, а не только размер кеша.

Сжатие памяти Windows: можно обойтись без записи на диск



Сжатие экономит RAM за счет CPU и откладывает чтение с диска.

Цена сжатия

Плюс

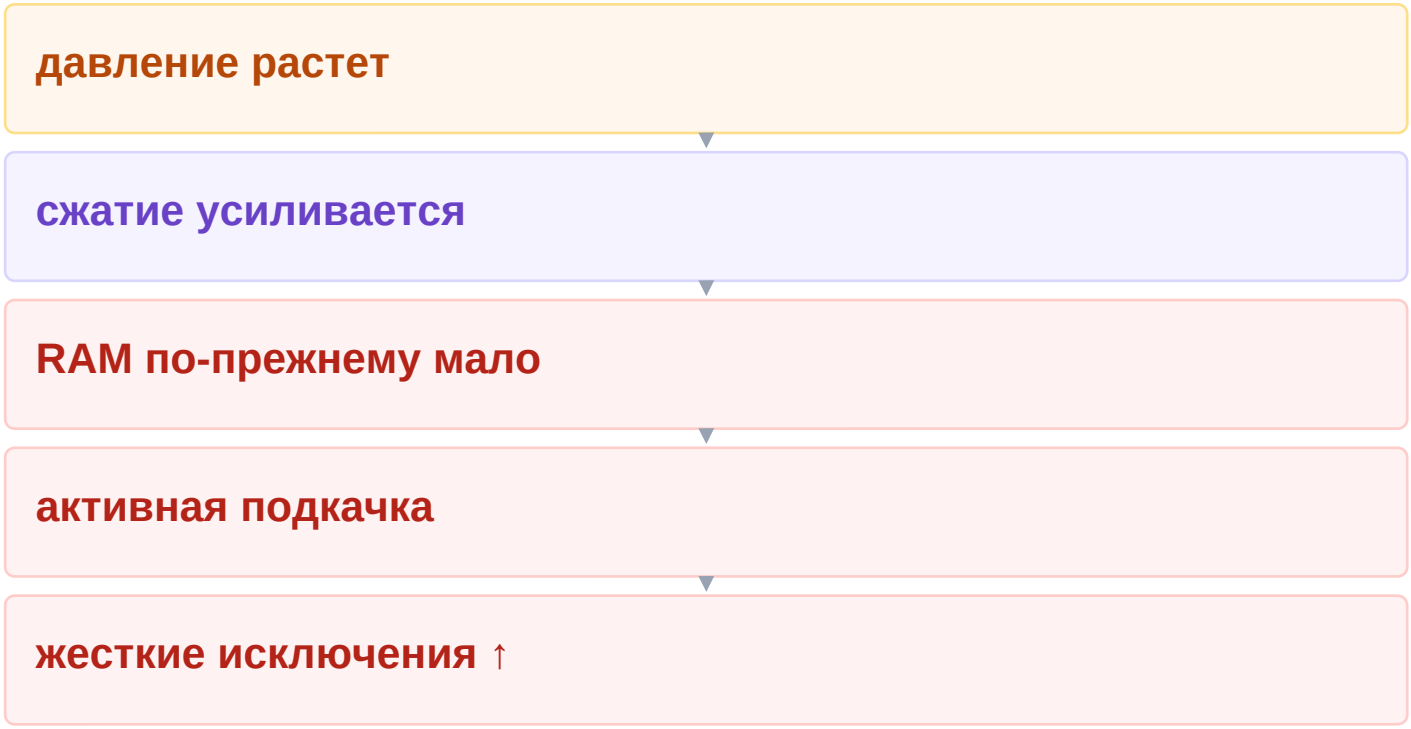
Меньше страниц в RAM и меньше обращений к файлу подкачки.

Цена

CPU тратится на сжатие и распаковку при повторном использовании.

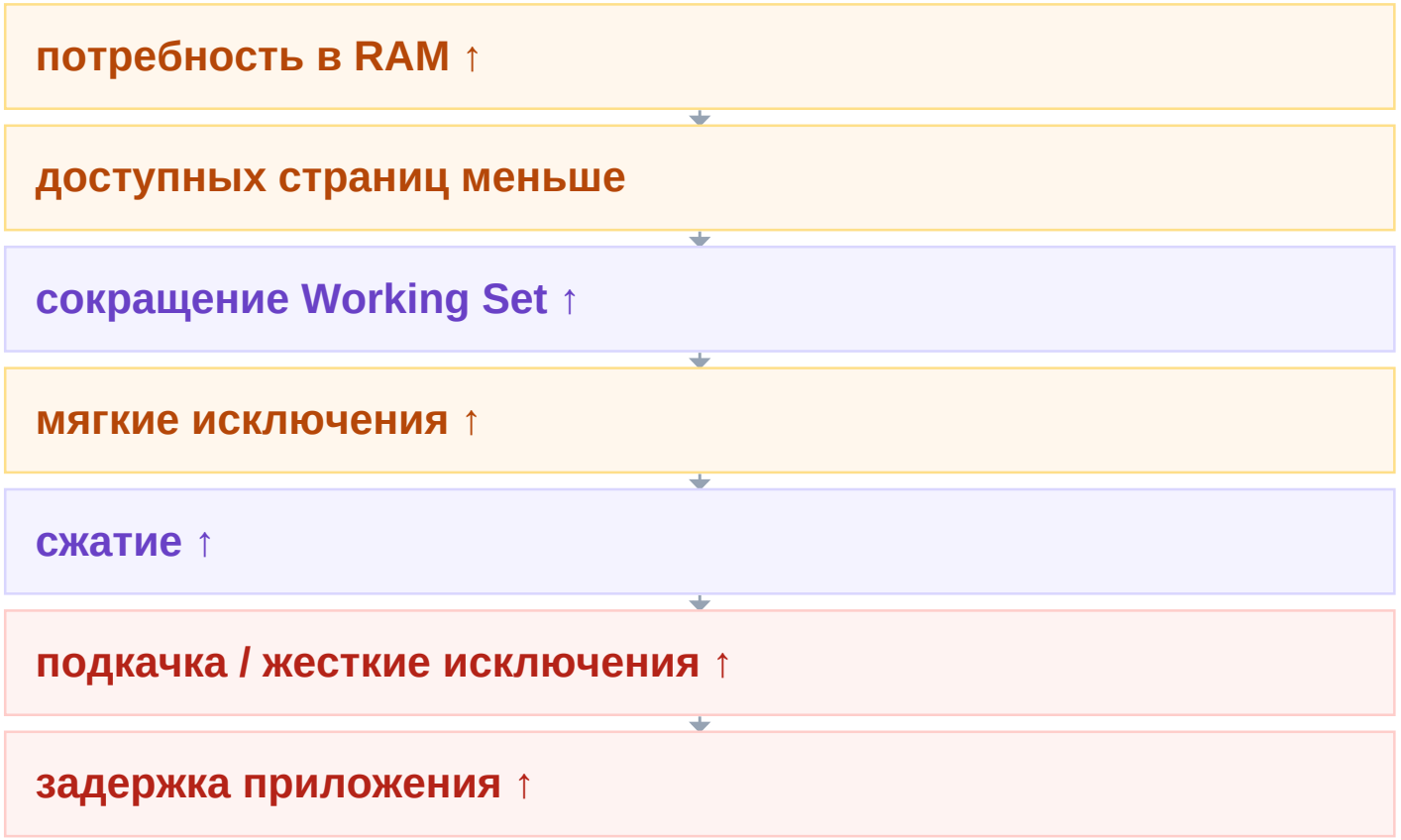
Это компромисс между RAM и CPU.

Сжатие не отменяет файл подкачки



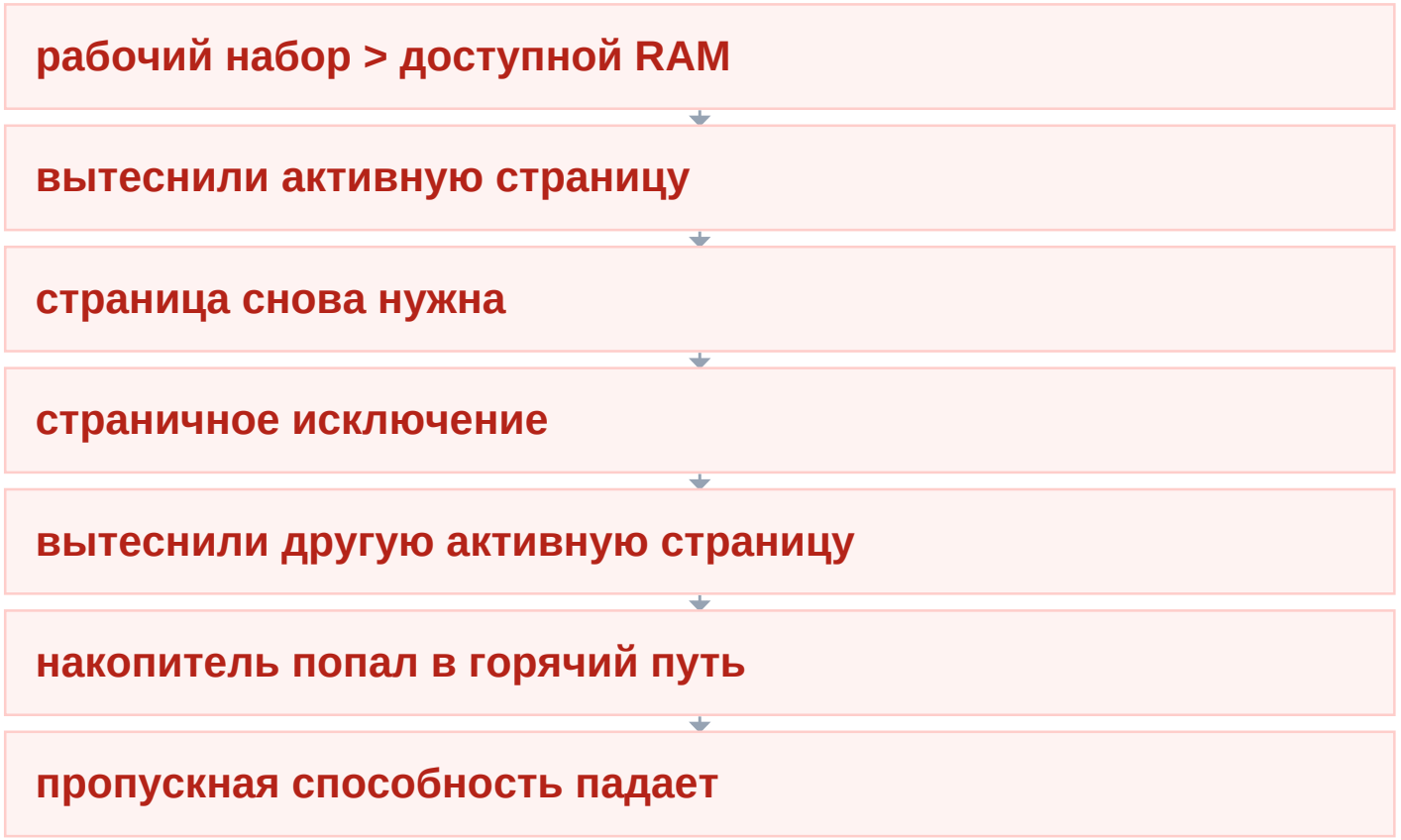
Сжатие — временный резерв, а не бесконечная память.

Упрощенная цепочка дефицита памяти Windows



Это причинная модель, а не жесткая последовательность состояний.

Постоянное вытеснение активных страниц в Windows



Когда активные страницы ходят между RAM и диском, полезная работа почти останавливается.

Дефицит Commit и физической RAM — разные проблемы

Дефицит Commit

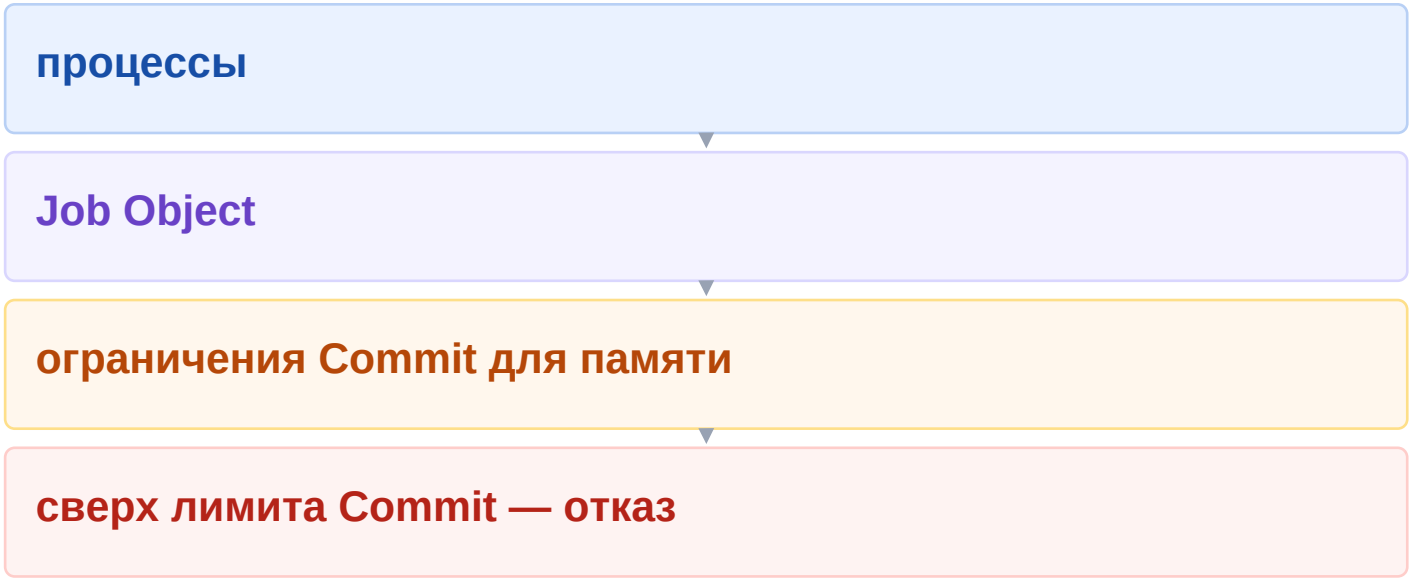
Система исчерпала обеспечение: Commit Charge → Commit Limit.

Дефицит физической RAM

Активный рабочий набор больше доступной физической RAM.

Сначала определяем, какую из двух осей мы реально уперли.

Job Objects: ограничения ресурсов группы процессов



По назначению Job Object похож на cgroup, но их механика различается.

Лимит Job не равен лимиту Working Set

Ограничиваемый ресурс

Ограничение относится к памяти с Commit процесса/Job.

Другая ось

Working Set отражает текущую резидентность и может меняться при давлении на память.

Модель cgroup нельзя автоматически переносить на Windows.

Контейнеры Windows: ментальная модель



Лимит памяти контейнера опирается на механизмы Windows.

Рантайм Go на Windows: явные вызовы VM API

```
sysReserveOS  → VirtualAlloc(MEM_RESERVE)

sysAllocOS    → VirtualAlloc(MEM_RESERVE | MEM_COMMIT)

sysUnusedOS   → VirtualFree(MEM_DECOMMIT)

sysUsedOS     → VirtualAlloc(MEM_COMMIT)

sysFreeOS     → VirtualFree(MEM_RELEASE)
```

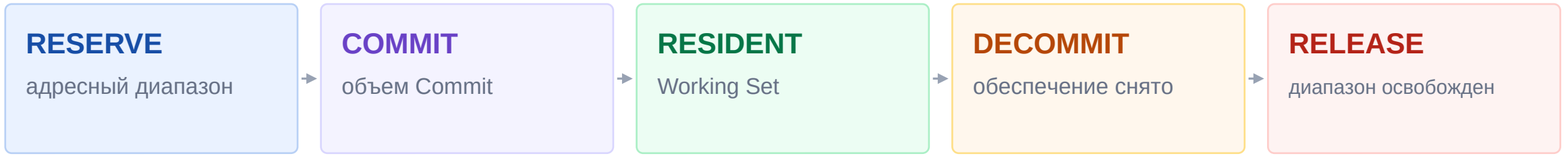
Слой памяти Go использует механизмы Windows: Reserve / Commit / Decommit / Release.

Что MEM_DECOMMIT означает для scavenger Go



Go может освободить физическую память, сохранив виртуальные адреса.

Windows дает почти идеальную лестницу состояний



Разделение этих уровней отлично объясняет, почему «процесс выделил память» - слишком грубая формулировка.

Диспетчер задач: одной колонки «Память» недостаточно

- Working Set?
- Commit / Private Bytes?
- системный Commit и Commit Limit?
- Доступная RAM?
- сжатая память?
- активность подкачки?

Нужно сначала сформулировать вопрос, затем выбрать счетчик.

PerfMon: основные счетчики

```
Memory\Committed Bytes
Memory\Commit Limit
Process\Working Set
Process\Private Bytes
Memory\Pages Input/sec
Memory\Pages Output/sec
```

PerfMon позволяет отделить Commit, резидентность и перемещение страниц.

Committed Bytes и Commit Limit

Committed Bytes

Общий объем памяти, обеспеченной системой.

Commit Limit

Максимально доступный Commit. Если показатели близки, подозреваем исчерпание Commit.

Это диагностика лимита Commit, а не Working Set.

Working Set и Private Bytes

Working Set

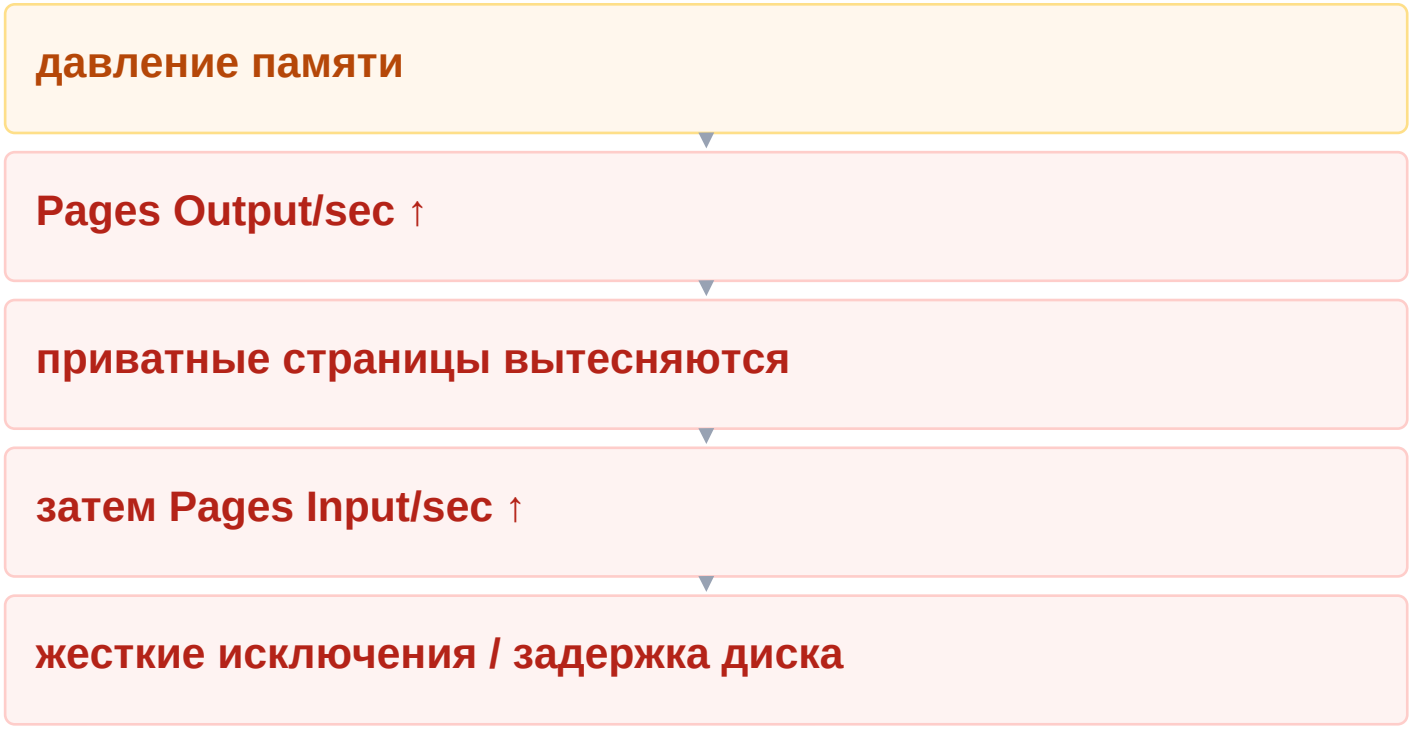
Страницы процесса в RAM сейчас; меняются при давлении.

Private Bytes

Приватная память с Commit; отражает скорее обязательства, чем количество страниц в RAM.

Рост Private Bytes и Working Set имеет разные причины.

Pages Input/Output: движение страниц между RAM и диском



Счетчики подкачки сопоставляем с задержкой диска, Working Set и p99.

RAMMap и VMMap отвечают на разные вопросы

RAMMap

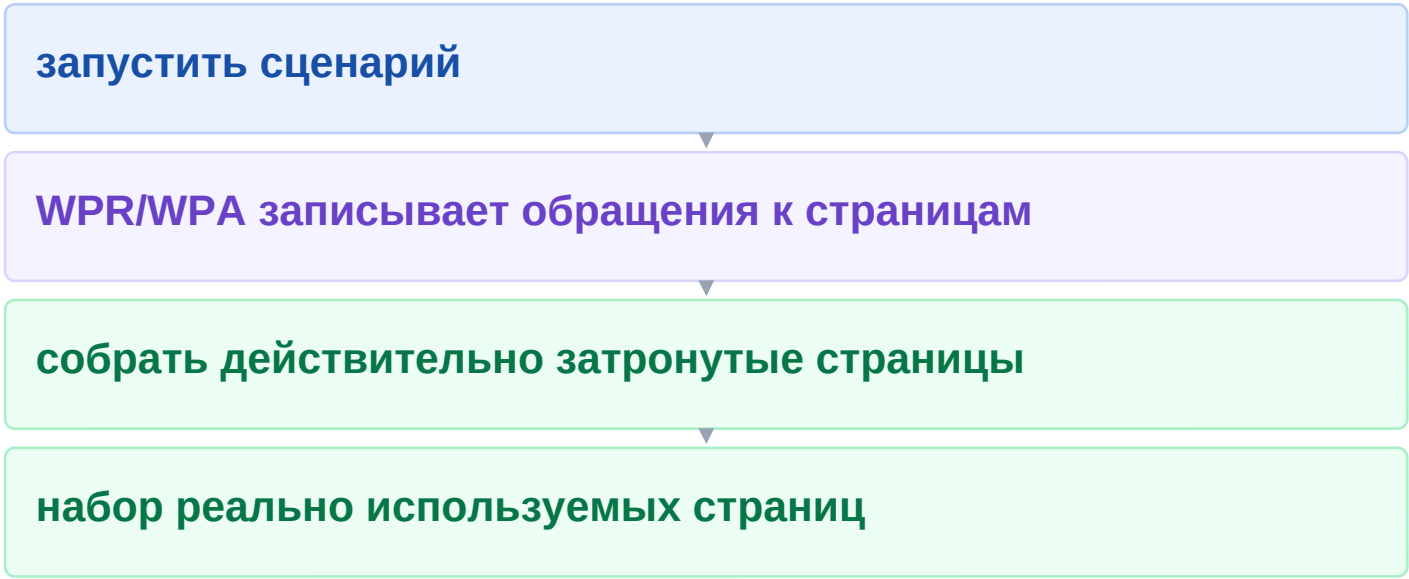
В RAM входят кеш файлов, рабочие наборы и списки страниц, и другие категории памяти.

VMMap / Process Explorer

Виртуальное пространство процесса: резерв, Commit, Commit, частная, отображенная, образ, Working Set.

Первый инструмент описывает систему, второй — конкретный процесс.

Reference Set: какие страницы реально затронул сценарий



Reference Set лучше отражает сценарий, чем снимок Working Set.

Windows: симптом → гипотеза №1

Наблюдение

Private Bytes стабильны, Working Set падает, мягкие исключения растут.

Гипотеза

Сокращение Working Set: Commit сохраняется, а резидентность уменьшается.

Данные процесса не исчезли - меняется их физическое размещение.

Windows: симптом → гипотеза №2

Наблюдение

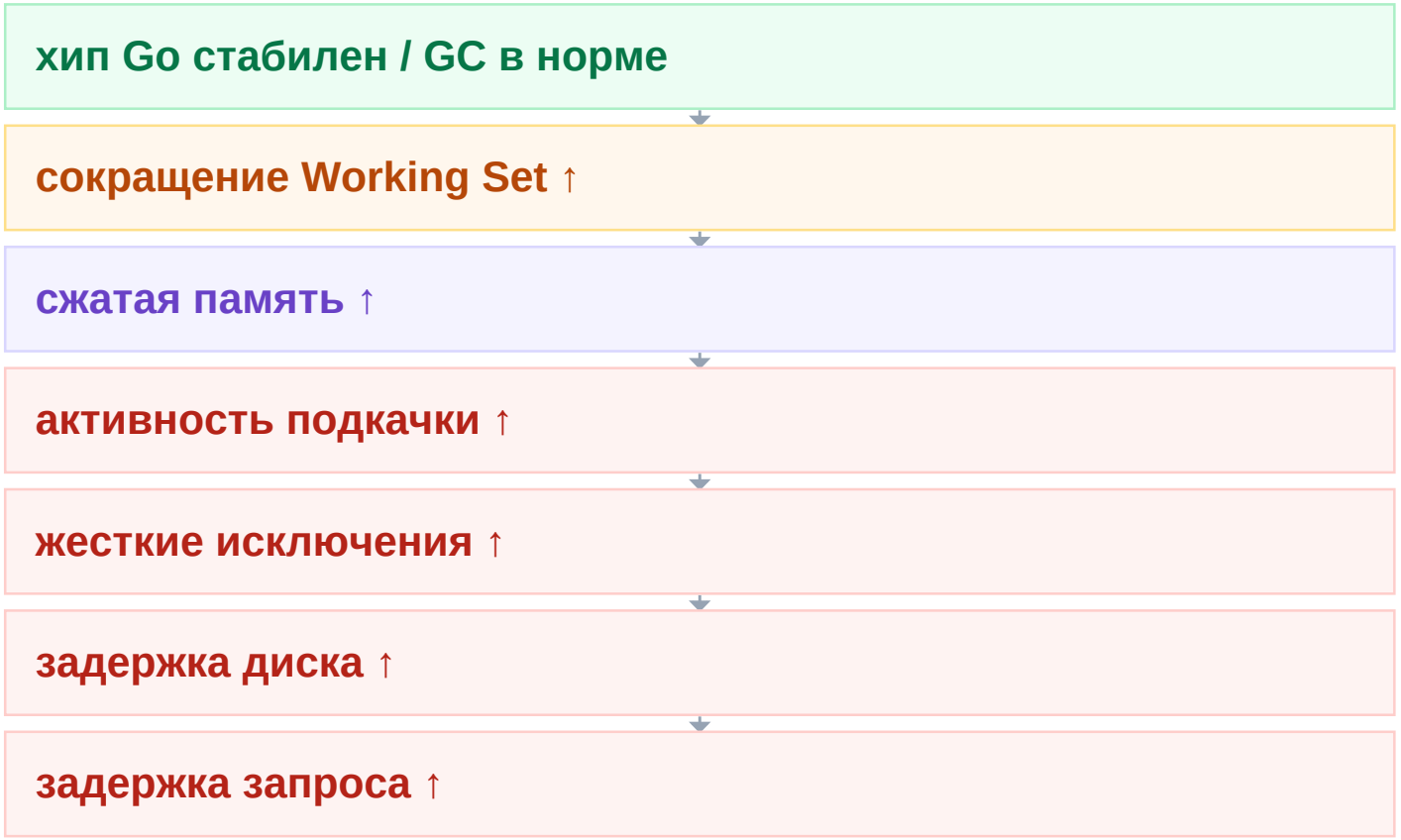
Committed Bytes близок к Commit Limit, новые выделения начинают завершаться ошибкой.

Гипотеза

Исчерпание Commit. Это отличается от постоянного вытеснения постоянным вытеснением страниц.

RAM и файл подкачки определяют запас Commit; способы решения другие.

Windows: давление на память при исправном GC Go



Backend может тормозить на уровне ОС при исправном рантайме Go.

Windows: что важно запомнить

- Reserve, Commit и Resident описывают разные уровни.
- Working Set — текущие резидентные страницы процесса.
- Commit — обеспечение памяти, а не занятая RAM.
- Мягкое исключение обходится без диска; жесткое требует чтения.
- Сокращение Working Set не освобождает адреса или Commit.
- Сжатие экономит RAM ценой CPU.
- Файл подкачки обеспечивает и подкачку, и запас Commit.
- Scavenger Go использует MEM_DECOMMIT / MEM_COMMIT / MEM_RELEASE.

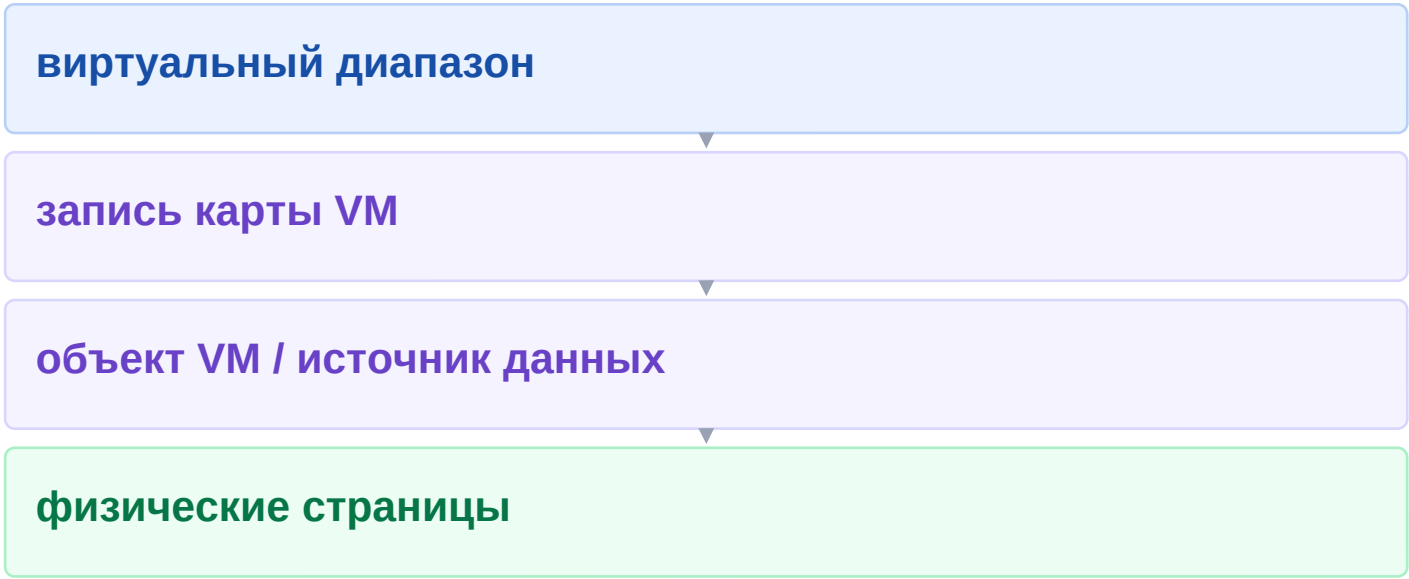
Windows наглядно разделяет адреса, обеспечение и резидентность.

macOS: та же VM, другая оценка давления памяти



Главный сводный индикатор здесь — Memory Pressure.

Mach VM: карта памяти и физическое размещение разделены



Для лекции важно видеть слои, а не уходить во внутренние структуры XNU.

VM map и pmap выполняют разные роли

VM map

Логическая карта виртуальных диапазонов процесса.

pmap

Низкоуровневая связь виртуальных и физических адресов.

Это разделяет карту адресов и реализацию таблиц страниц.

Выделение по требованию: виртуальный объем не равен RAM



Резидентность появляется по мере фактического доступа.

Резидентная память macOS — часть виртуального пространства

Виртуальная память

Большой виртуальный диапазон может существовать без полной резидентности.

В RAM

Физические страницы, которые сейчас находятся в RAM.

Один размер виртуальной памяти плохо описывает расход RAM.

Wired Memory: страницы, закрепленные в RAM

закрепленная память

▼

должна оставаться в RAM

▼

меньше RAM доступно остальным задачам

Закрепленные страницы уменьшают доступный запас RAM.

App Memory и подробная диагностика процесса

Activity Monitor

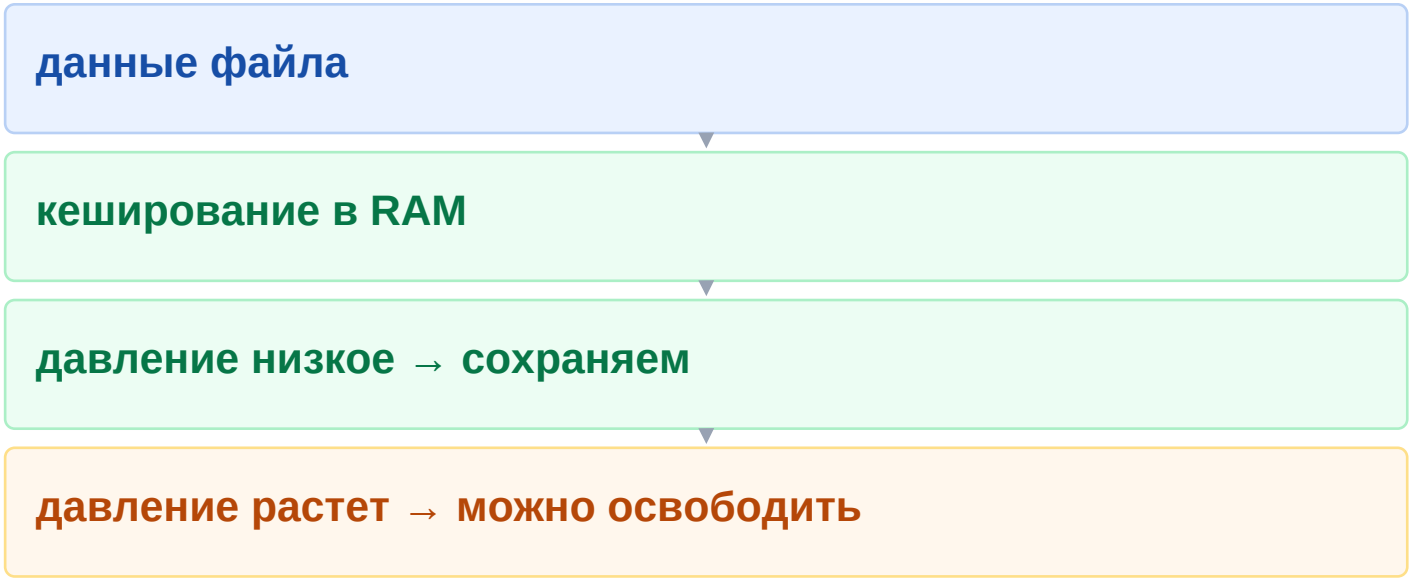
App Memory - удобная агрегированная метрика для обзора.

Глубже

vmmap, Instruments, vm_stat помогают понять отображения / резидентность / память процесса.

Интерфейс мониторинга и инструменты VM отвечают на разные вопросы.

Cached Files: свободная RAM работает как кеш



Небольшой объем Free Memory не означает дефицит.

Memory Pressure важнее одной Free Memory

Free Memory

Объем полностью свободной RAM в данный момент.

Memory Pressure

Сводная оценка того, насколько эффективно система обеспечивает запросы с учетом кеша, закрепленных страниц, сжатия и свопа.

На macOS важнее давление памяти, чем просто объем свободной RAM.

Зеленый / желтый / красный Memory Pressure

Зеленый

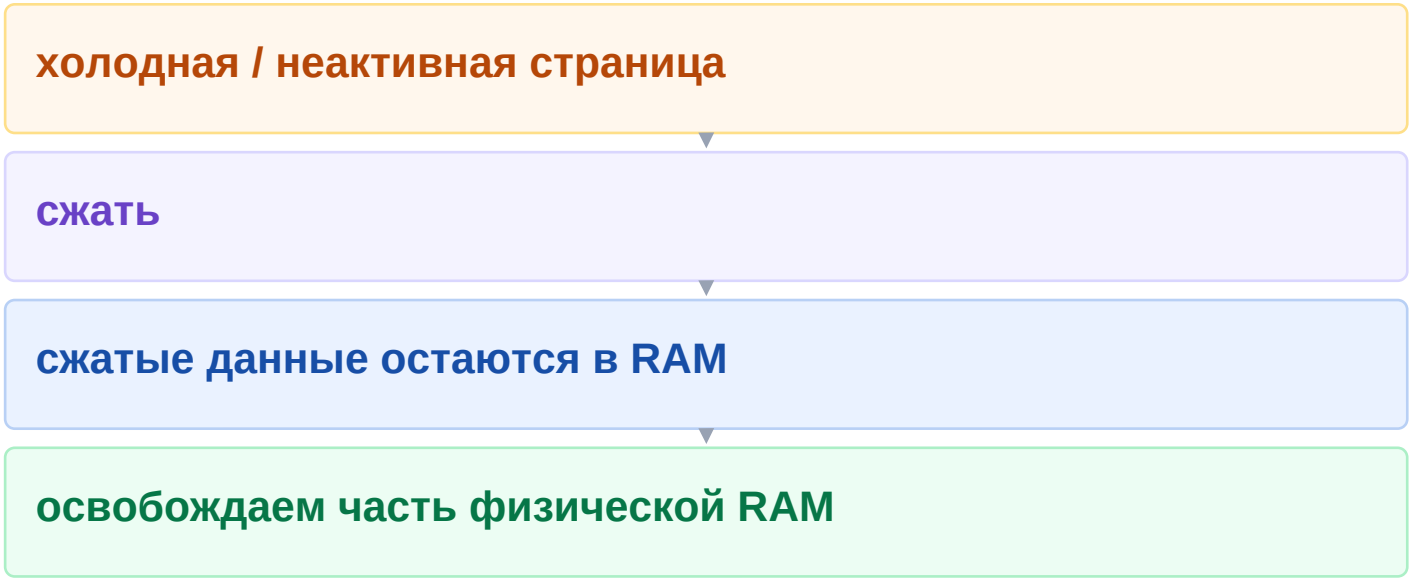
Система справляется с памятью; небольшая свободная RAM может быть нормальна.

Желтый / красный

Давление растет → сжатие, своп и освобождение страниц становятся дороже и мешают работе.

Цвет — сводный показатель, а не точная модель состояний ядра.

Сжатие macOS: хранить данные в RAM плотнее



Сжатие откладывает обращения к диску ценой CPU.

Почему сжатие действительно экономит RAM

До

логическая страница 4 КиБ
занимает около 4 КиБ физической RAM



После

условно 1 КиБ сжатых данных
остается в RAM

Экономим RAM, но тратим CPU на сжатие и распаковку.

Сжатие = RAM ↔ CPU

Пока страница холодная

Сжатые данные дешевле чтения с диска и занимают меньше RAM.

Когда страница снова нужна

Распаковка при повторном доступе увеличивает задержку запроса.

Сжатие обычно дешевле обращения к SSD, но не бесплатно.

Compressed Memory продолжает занимать RAM

Плохая модель

Compressed = 4 GiB → «освободили 4 GiB RAM».

Точная модель

Часть логической памяти хранится в сжатом виде внутри RAM.

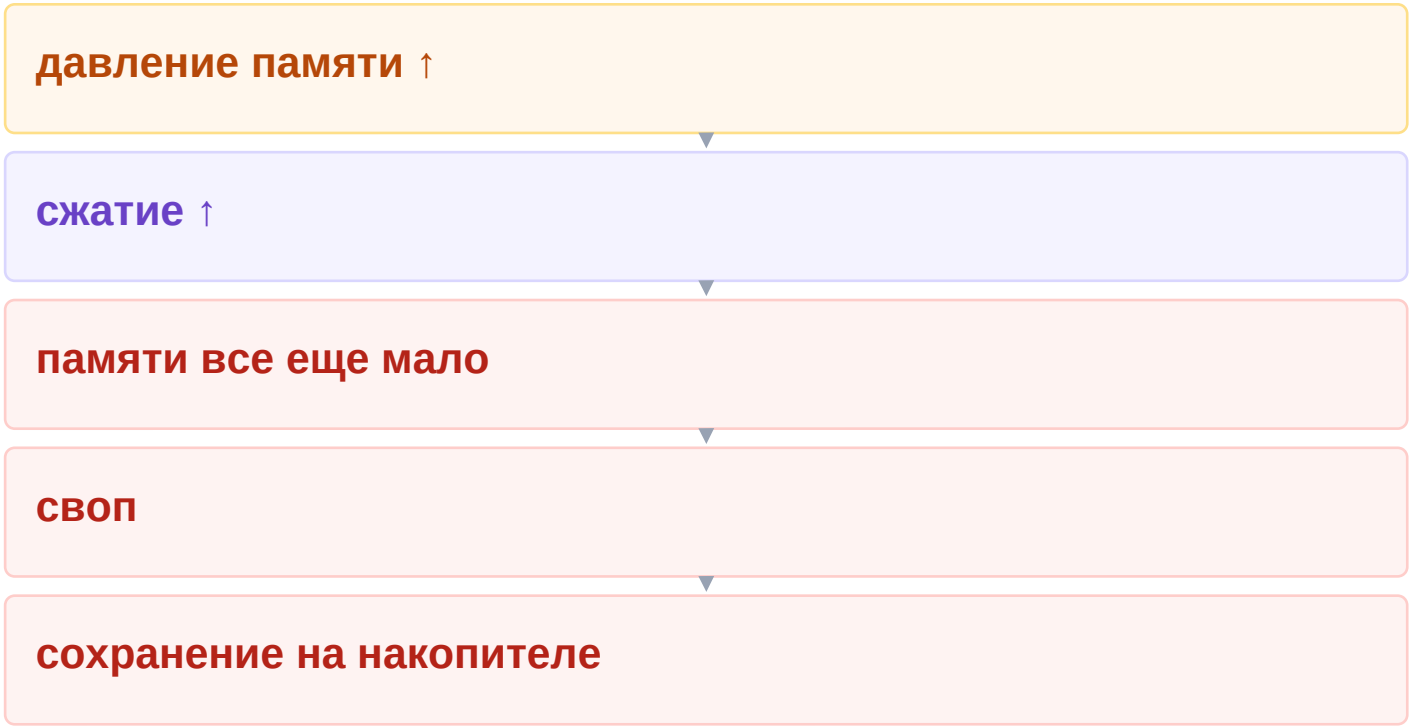
Сжатые данные продолжают занимать RAM, в отличие от Free Memory.

Когда сжатие само становится заметной нагрузкой



Частые сжатия и распаковки активных страниц расходуют CPU.

Своп macOS: когда сжатия недостаточно



Своп подключает накопитель, который намного медленнее RAM.

Swap Used > 0 — еще не проблема

Может быть нормально

Холодные страницы давно в свопе и почти не используются.

Плохо

Своп активен, страницы часто возвращаются; уровень Memory Pressure высокий, задержки растут.

Важна активность свопа, а не только его занятый объем.

Возврат страницы из свопа



Даже быстрый NVMe намного медленнее DRAM: активный своп повышает p95/p99.

Apple Silicon не отменяет давление на память

Что меняется

CPU, GPU и другие блоки используют общую физическую RAM; требуется меньше копирований между областями памяти.

Что остается

Емкость RAM конечна: сжатие, своп, давление памяти и вытеснение страниц никуда не исчезают.

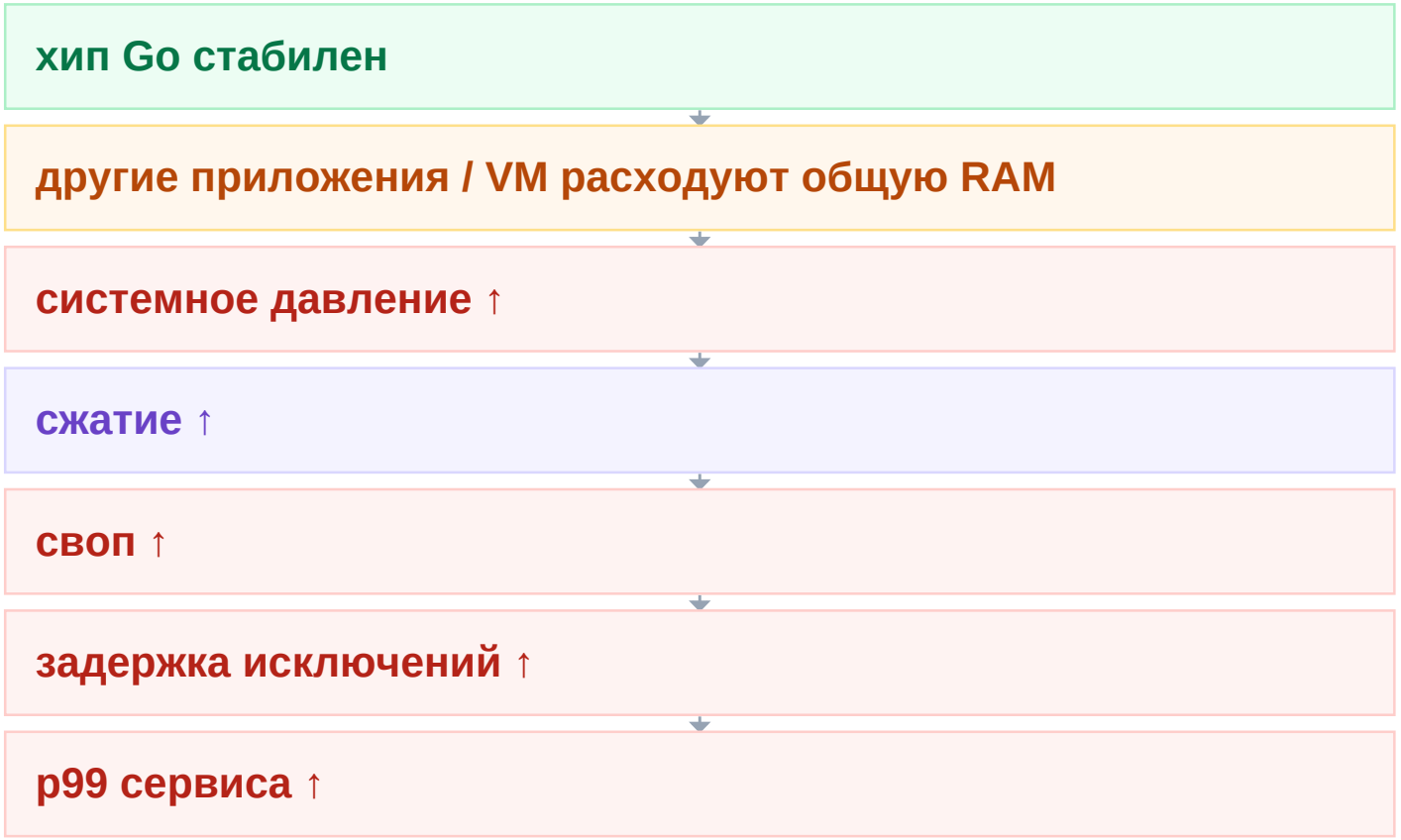
Unified Memory меняет устройство памяти, но не увеличивает ее емкость.

Unified Memory: больше потребителей одной физической RAM

- процесс Go;
- нагрузка GPU;
- обработка видео;
- браузер и IDE;
- Docker / виртуальная машина;
- системные службы и кеши.

Go-сервис на Mac может тормозить из-за нагрузки других приложений.

Почему локальный Go-сервис на Mac может внезапно замедлиться



Локальный тест зависит от состояния всей машины, не только процесса Go.

Сбрасываемая память: кеш должен допускать восстановление

Восстанавливаемый кеш

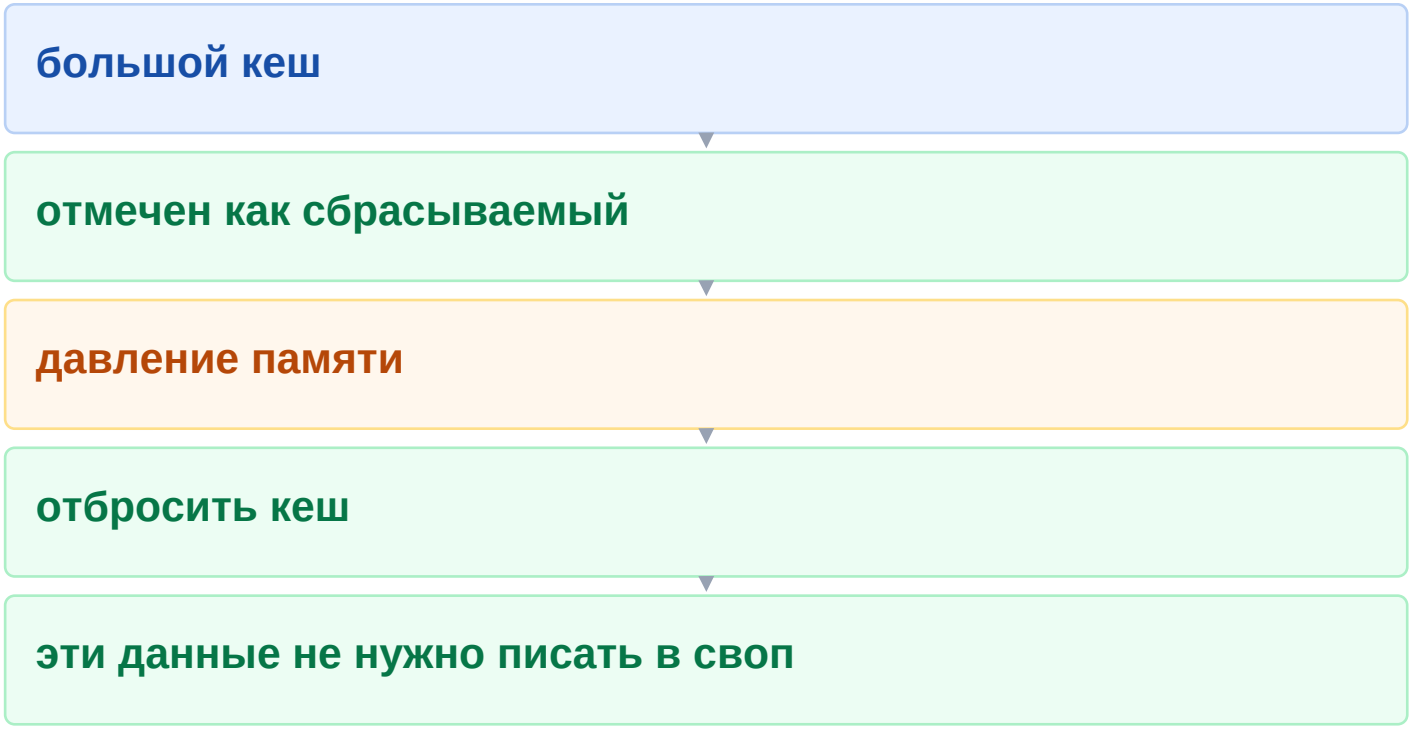
При дефиците памяти данные можно отбросить и восстановить.

Уникальные изменяемые данные

Данные нельзя отбросить: содержимое надо сохранить.

Восстанавливаемые данные дают диспетчеру памяти больше свободы.

Почему сбрасываемый кеш дешевле при дефиците RAM



Архитектура может сделать часть данных удобными для освобождения.

Своп и износ SSD: что действительно важно

Плохая формула

«Любой своп убивает SSD».

Что измеряем

Реальный объем записей, их частоту, Memory Pressure, нагрузку и задержкой.

Постоянный активный своп означает, что рабочий набор не помещается в RAM.

vm_stat: системная динамика VM

```
vm_stat 1

free / active / inactive / speculative
wired / compressed
pageins / pageouts
страничные исключения
```

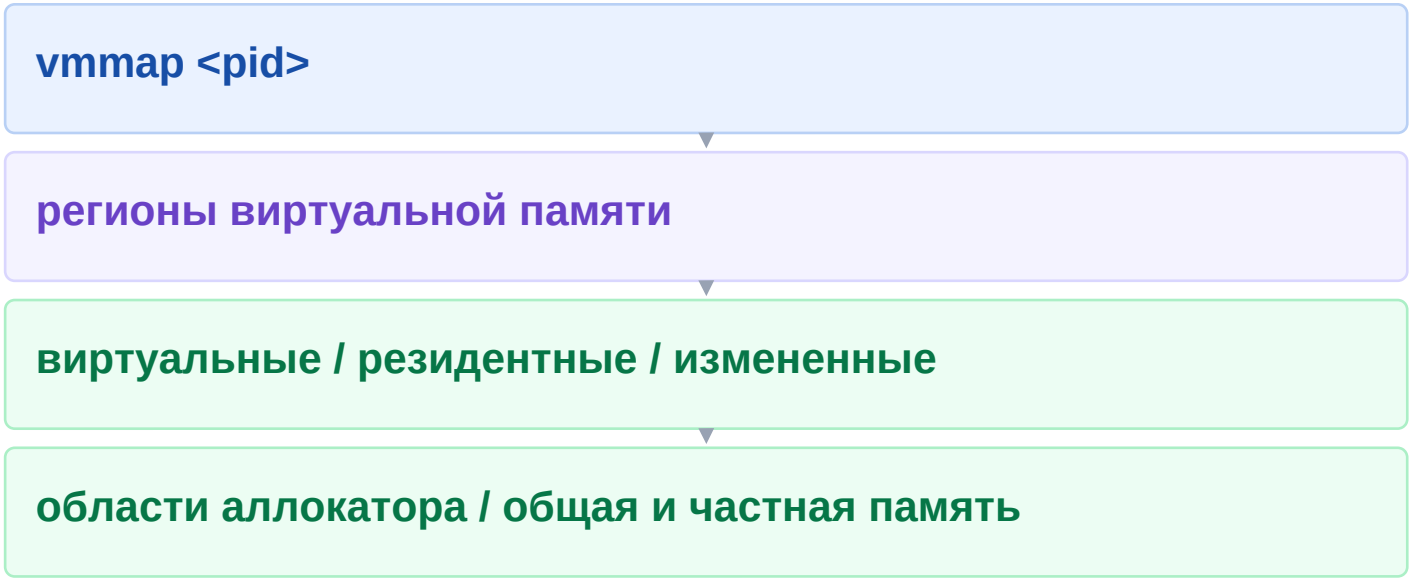
Одна строка мало говорит — нужна динамика счетчиков.

memory_pressure: сводная оценка состояния системы



Помогает отличить общий дефицит RAM от проблемы одного процесса.

vmmap: изучить адресное пространство процесса



vmmap показывает устройство виртуальной памяти процесса macOS.

Activity Monitor: первый уровень диагностики

- Memory Pressure;
- Physical Memory / Memory Used;
- App Memory;
- Wired Memory;
- Compressed;
- Cached Files;
- Swap Used.

Activity Monitor показывает состояние всей системы, а не одного процесса.

Instruments дополняет Go pprof

Инструменты Go

pprof и runtime/metrics показывают хип, GC, аллокации и CPU рантайма.

Инструменты Apple

Allocations, Leaks, VM Tracker и Time Profiler показывают нативную память процесса и отображения VM.

При проблеме с памятью нужны измерения на обоих уровнях.

Рантайм Go на macOS: API виртуальной памяти Darwin

```
sysAllocOS    → mmap(PROT_READ|PROT_WRITE, MAP_ANON|MAP_PRIVATE)

sysUnusedOS   → madvise(MADV_FREE_REUSABLE)

sysUsedOS     → madvise(MADV_FREE_REUSE)

sysFreeOS     → munmap
```

Scavenger Go использует механизмы повторного использования памяти Darwin.

MADV_FREE_REUSABLE: виртуальный адрес остается, RAM можно вернуть



Рантайм возвращает физическую память, сохраняя виртуальный диапазон.

MADV_FREE_REUSE: диапазон снова нужен Go



API Darwin явно разделяет освобождение и повторное использование памяти.

Scavenger Go и сжатие macOS — разные механизмы

Scavenger

Go уже знает: эти страницы хипа не нужны. Можно вернуть физических страниц.

Compression

OS считает данные все еще нужными, но холодными, и хранит их плотнее.

Первое определяет владение памятью, второе — резидентность страниц.

Почему это разделение важно

Объект умер

GC, sweep и scavenger могут освободить объекты и емкость памяти.

Объект жив, но неактивен

GC сохраняет объект; ОС может лишь сжать или выгрузить физических страниц.

Достижимость и активность — разные свойства.

Упрощенная цепочка давления памяти macOS

потребность в RAM ↑

↓

освобождаемого кеша меньше

↓

сжатие ↑

↓

давление продолжает расти

↓

своп ↑

↓

распаковка / возврат страниц / ввод-вывод ↑

↓

задержка приложения ↑

Это причинная схема, не строгий автомат состояний.

Постоянное вытеснение активных страниц в macOS

активный набор > физической RAM

▼

постоянное сжатие / распаковка

▼

постоянная запись / чтение свопа

▼

нагрузка на CPU и диск ↑

▼

Memory Pressure: желтый / красный

▼

пропускная способность ↓

Если рабочий набор не помещается в RAM, сжатие лишь откладывает предел.

Почему Mac может использовать почти всю RAM и быть быстрым

RAM почти полностью занята

Приложения, кеш и файлы занимают RAM, потому что свободная RAM сама по себе не приносит пользы.

Здоровое состояние

Система может быстро освободить память; Memory Pressure остается зеленым.

Высокое использование RAM без давления — нормальная работа VM.

macOS: что важно запомнить

- Mach VM разделяет виртуальные отображения и физическую RAM.
- Закрепленная память уменьшает объем вытесняемых страниц.
- Memory Pressure информативнее объема свободной памяти.
- Сжатые данные продолжают занимать RAM.
- Сжатие экономит RAM ценой CPU и откладывает своп.
- Swap Used > 0 сам по себе не означает проблему.
- Unified Memory не отменяет дефицит RAM.
- Scavenger Go использует MADV_FREE_REUSABLE / MADV_FREE_REUSE.

Память macOS оцениваем по давлению, сжатию и активности свопа.

СРАВНЕНИЕ

Windows и macOS: две разные модели эксплуатации

Windows

Commit
Working Set
мягкие и жесткие исключения
сжатие
файл подкачки
PerfMon / RAMMap / WPA

macOS

Memory Pressure
резидентные / закрепленные
сжатие
своп
vm_stat / vmmap
Activity Monitor

Общий физический слой один, но эксплуатационный язык различается.

Сравнительная таблица: одна задача - разные интерфейсы

Вопрос	Windows	macOS
Виртуальный диапазон	MEM_RESERVE	отображение VM / mmap
Обязательство обеспечения	Commit	обеспечение Mach/Darwin VM
Резидентные страницы процесса	Working Set	резидентная память
Уменьшить резидентность	сокращение Working Set	освобождение / сжатие / подкачка
Хранение приватных данных на диске	файл подкачки	своп
Сжатие	Memory Compression	Compressed Memory
Общая оценка давления	Commit / доступная RAM / подкачка	Memory Pressure
Go возвращает физическую память	MEM_DECOMMIT	MADV_FREE_REUSABLE

Сравниваем функциональные роли, а не одинаково звучащие названия.

СРАВНЕНИЕ

Commit Windows и Memory Pressure macOS — разные показатели

Windows Commit

Учет обеспеченной памяти. Отвечает: сколько памяти система уже пообещала.

macOS Memory Pressure

Сводный показатель состояния памяти. Отвечает: насколько системе трудно удовлетворять запросы.

Они находятся на разных уровнях модели.

СРАВНЕНИЕ

Общая модель Working Set

Windows

Working Set — официальный термин для резидентных страниц процесса.

macOS

Удобно говорить «резидентный рабочий набор»: это модель действительно используемых страниц в RAM.

Активный рабочий набор важнее общего объема виртуальных адресов.

СРАВНЕНИЕ

Общая модель сжатия памяти

Windows

Холодные данные можно хранить сжатыми в RAM, уменьшая дисковой подкачки.

macOS

Сжатие — один из основных механизмов управления дефицитом памяти до свопа или вместо него.

Общий компромисс: меньше RAM ↔ больше CPU.

СРАВНЕНИЕ

Общая модель файла подкачки и свопа

файл подкачки Windows

Обеспечение части приватной памяти + участие в Commit емкости.

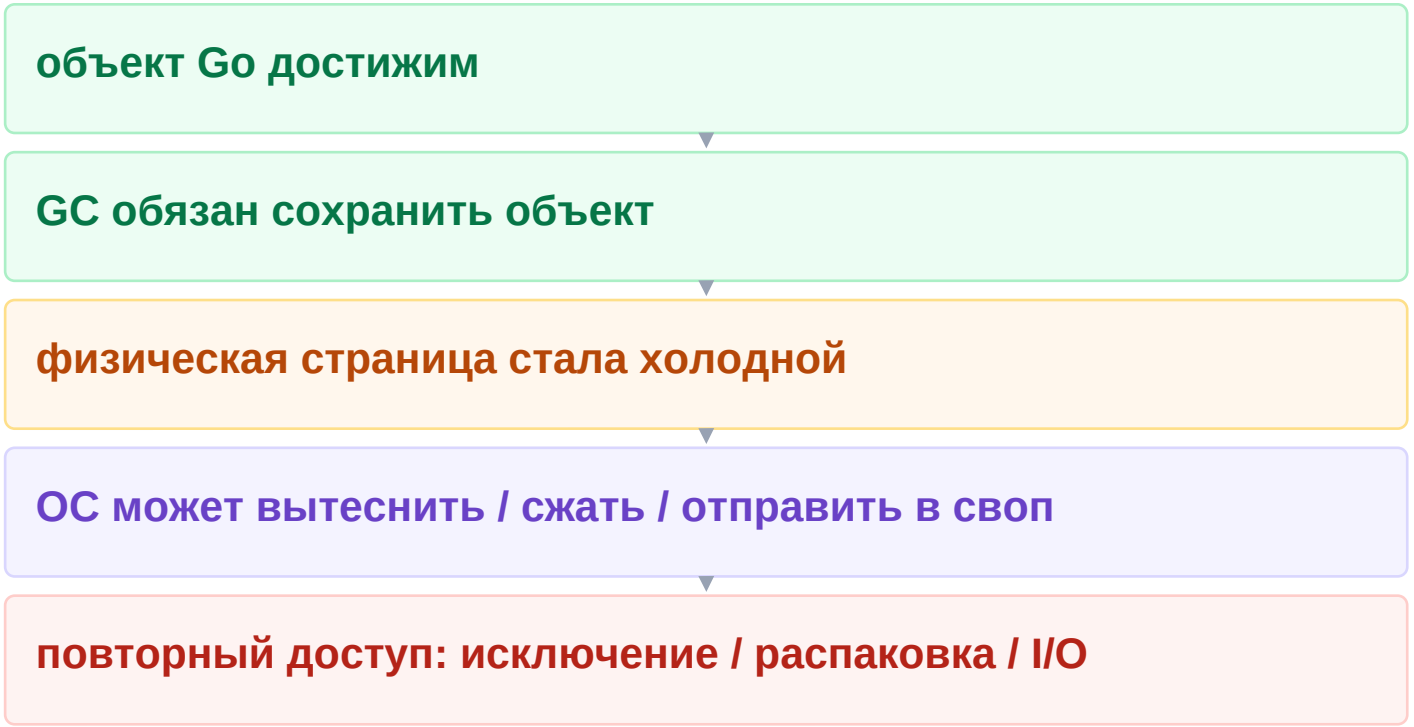
своп macOS

Сохранение вытесненных страниц на накопителе.

Возврат данных с диска повышает задержку в обеих ОС.

СРАВНЕНИЕ

Достижимость в Go и активность страниц ОС — разные вещи



GC спрашивает «объект достижим?», OS спрашивает «страница активна прямо сейчас?».

СРАВНЕНИЕ

Достижимый объект может быть холодным

рантайм Go

Не может удалить объект: на него есть путь от корней GC.

диспетчер памяти ОС

Может вытеснить страницу из RAM, если она давно не использовалась.

Поэтому живой хип и резидентный рабочий набор различаются.

Сквозная цепочка Windows



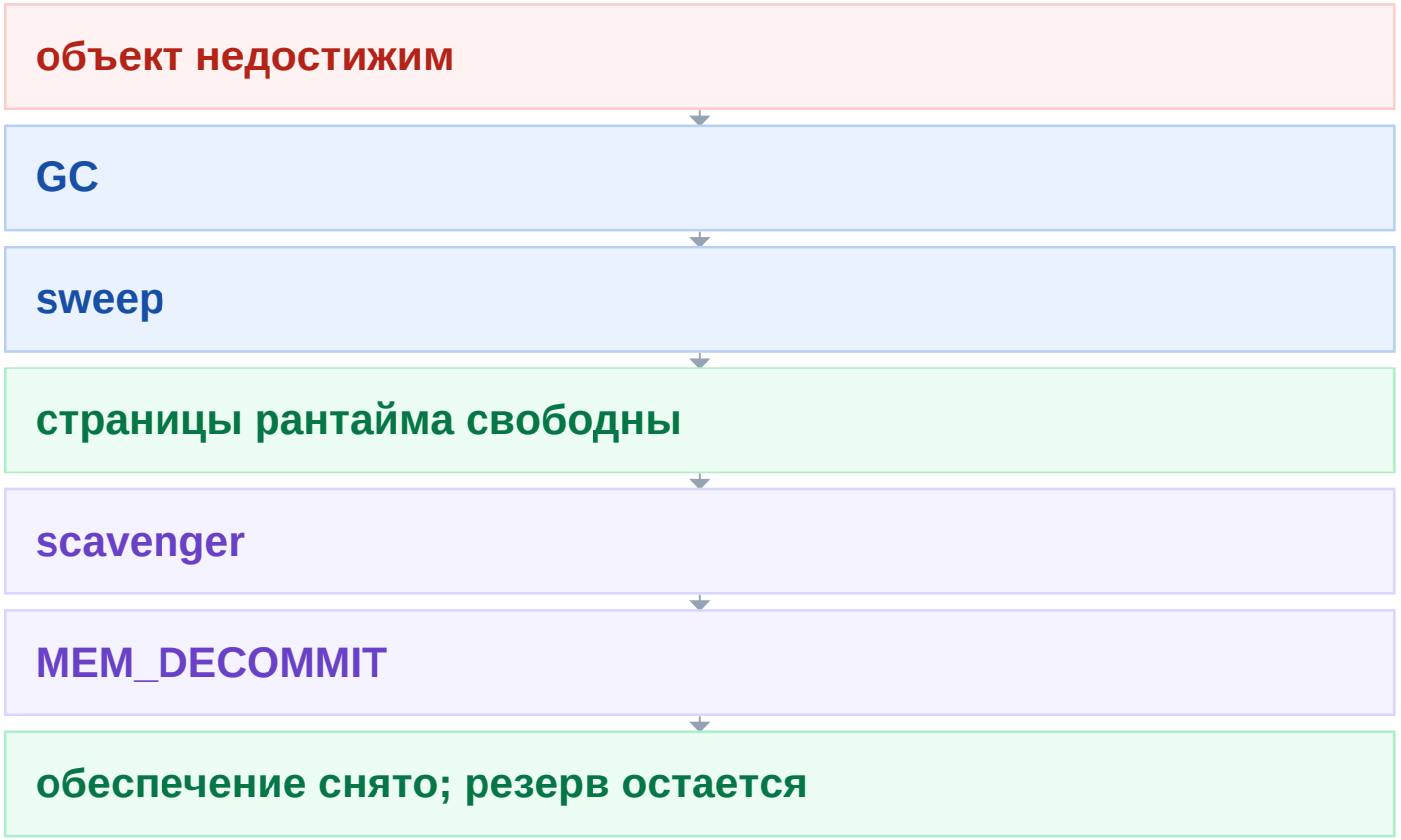
Go сохраняет объект; Windows независимо управляет страницами в RAM.

Сквозная цепочка macOS



Даже при исправном GC возврат холодных страниц может быть дорогим.

Scavenger Go на Windows



Если Go не нужны страницы, их можно вернуть и без дефицита RAM в ОС.

Scavenger Go на macOS



Одна задача рантайма реализуется разными API виртуальной памяти.

Как выглядит здоровая Windows-система

Commit далеко от лимита

↓

Working Set стабилен

↓

жестких исключений мало

↓

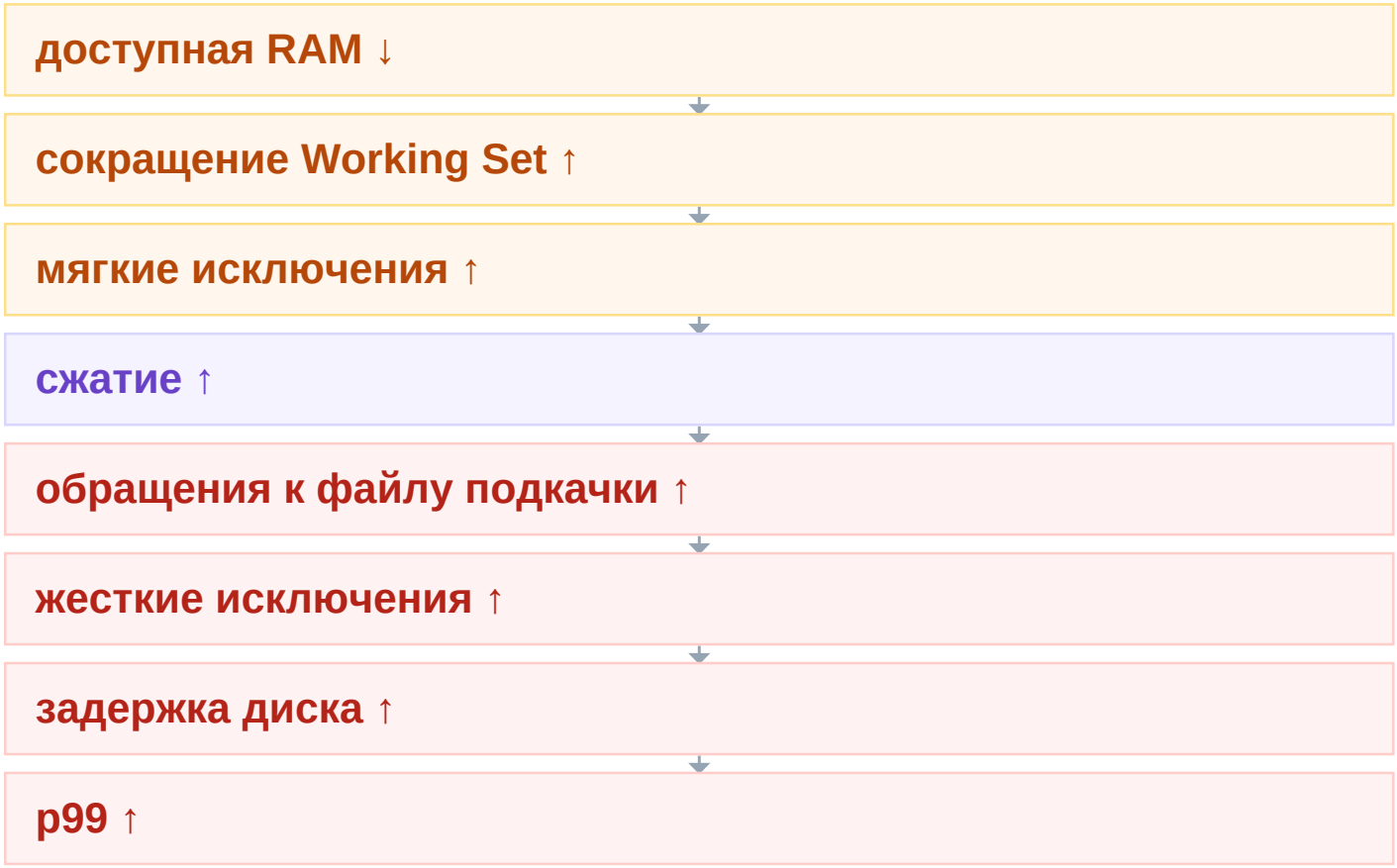
обращений к файлу подкачки мало

↓

задержка приложения стабильна

RAM может быть сильно занята - это само по себе не проблема.

Признаки нехватки памяти Windows



Отказ в выделении памяти может быть далеко, но задержки уже растут.

Как выглядит здоровая macOS

Memory Pressure: зеленый

↓

сжатие умеренное / стабильное

↓

активность свопа мала / стабильна

↓

задержка приложения стабильна

Малый объем Free Memory при этом может быть нормальным.

Признаки нехватки памяти macOS

Memory Pressure: желтый / красный

▼

сжатая память ↑

▼

активность свопа ↑

▼

распаковка / возврат страниц ↑

▼

нагрузка на CPU и диск ↑

▼

задержка приложения ↑

Смотрим давление и активность свопа, а не только объем Swap Used.

Стабильный хип Go не исключает проблему с памятью ОС

Go

живой хип стабилен, GC CPU низкий, паузы короткие.

OS

Windows: вытеснение / подкачка / жесткие исключения ↑
macOS: давление / сжатие / своп ↑

Причина проблемы с памятью может быть ниже рантайма Go.

Локальный тест искажается дефицитом RAM на всей машине

Mac

IDE + браузер + Docker VM + видео → давление общей памяти → сжатие / своп → колебания задержки.

Windows

Defender + браузер + IDE + Docker/VM + файловый кеш → Working Set / изменения подкачки.

Тест памяти проводим при контролируемом состоянии ОС.

Практика Windows: Reserve → Commit → Resident



Руками доказать: резерв ≠ Commit ≠ страницы в RAM.

Практика Windows: сокращение Working Set

- создать процесс с большим Working Set;
- другая нагрузка создает дефицит RAM;
- наблюдать Working Set;
- наблюдать мягкие / жесткие исключения;
- наблюдать активность файла подкачки;
- сопоставить с задержкой.

Владение памятью и число страниц в RAM меняются по-разному.

Практика Windows: сжатие и подкачка

- создать управляемый дефицит RAM;
- смотреть Диспетчер задач / PerfMon / RAMMap;
- сравнить сжатие, подкачку, жесткие исключения и задержку;
- не ожидать единого порога: алгоритм адаптивный.

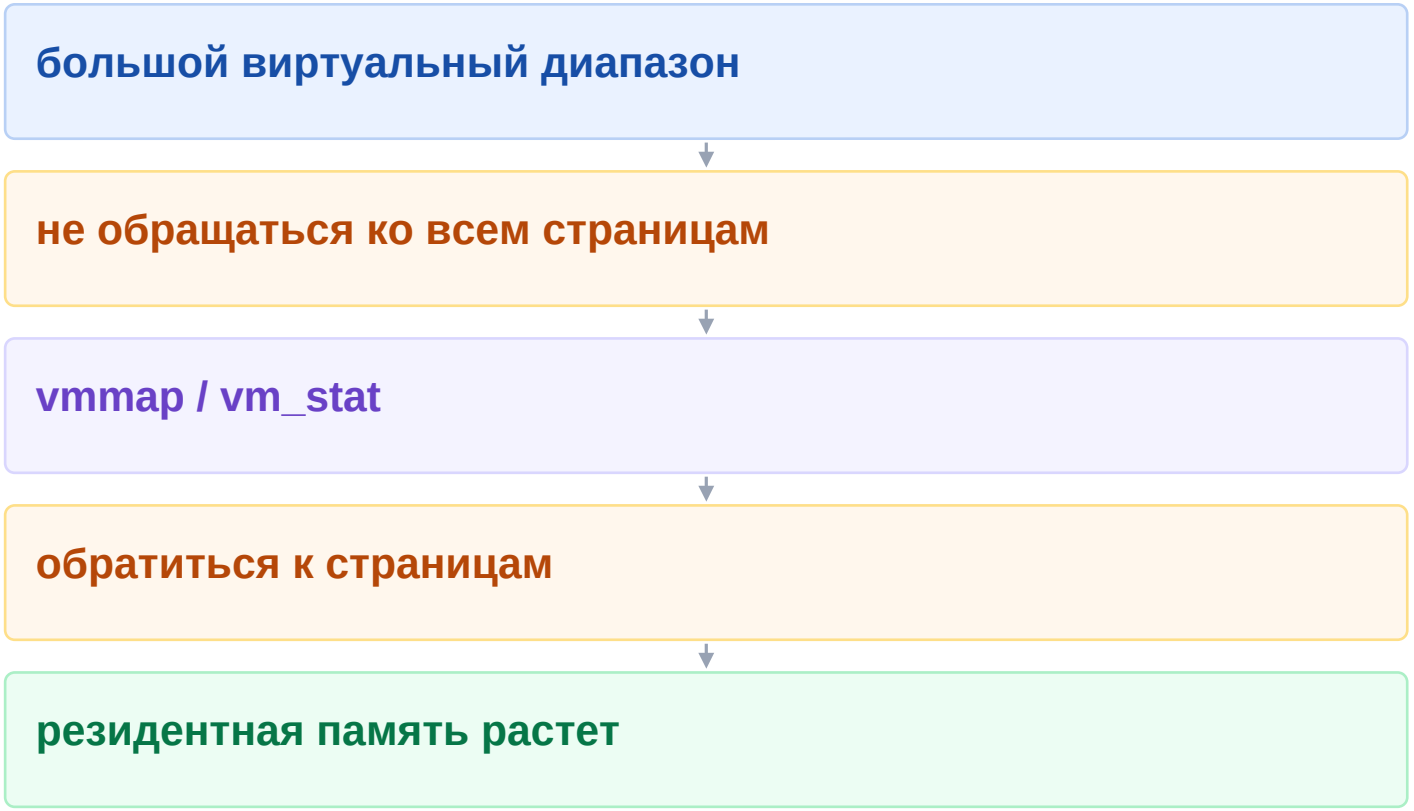
Цель — увидеть переход от страниц в RAM к сжатию и чтению с диска.

Практика Windows: Commit Limit



Исчерпание Commit и дефицит RAM — два разных эксперимента.

Практика macOS: виртуальный диапазон → первый доступ



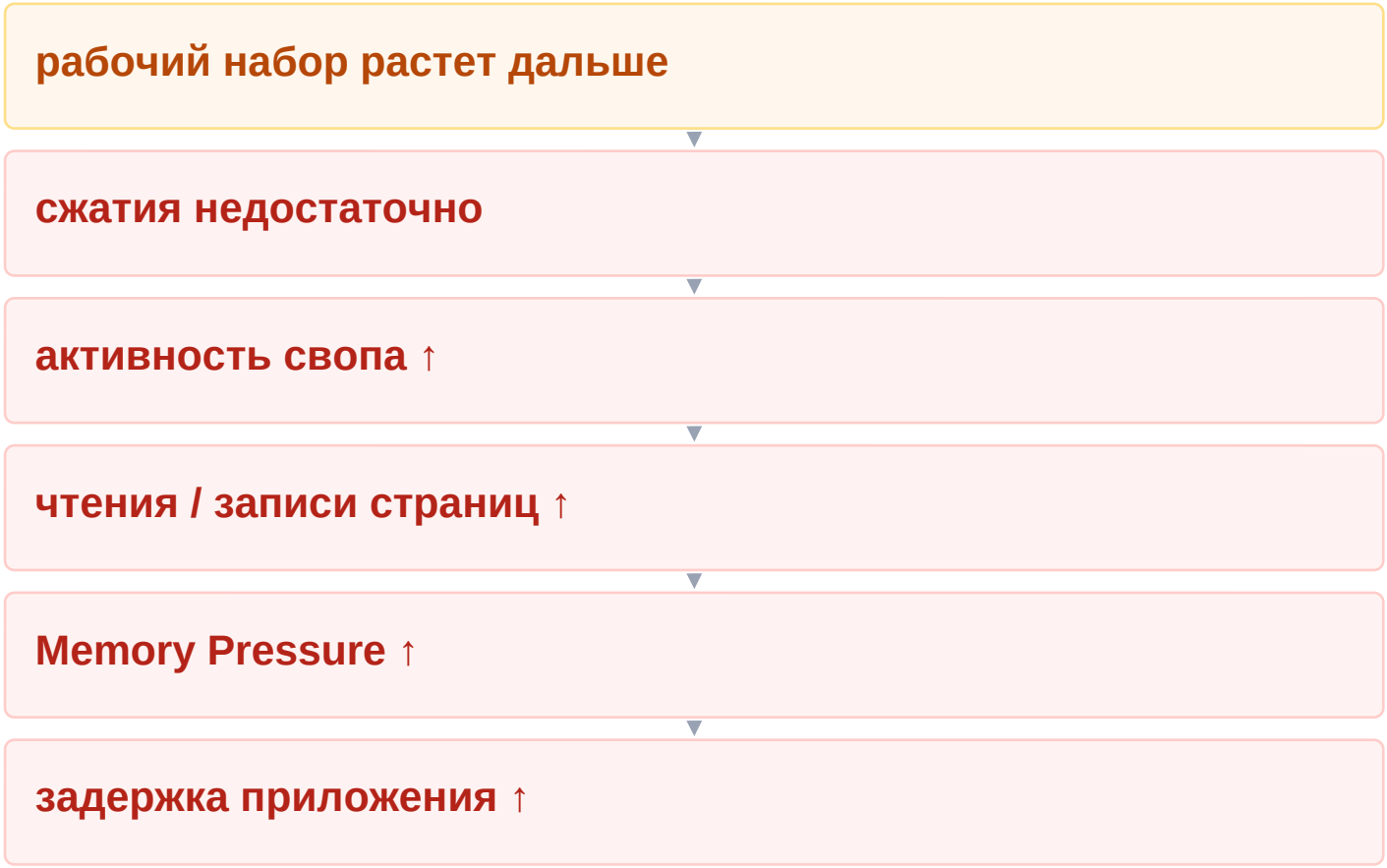
Руками доказать: виртуальный диапазон ≠ занятая RAM.

Практика macOS: сжатие

- создать управляемый дефицит памяти;
- Activity Monitor: Memory Pressure, Compressed, Swap Used;
- vm_stat 1;
- memory_pressure;
- параллельно измерять задержку обработки.

Показать переход от страниц в RAM к сжатию.

Практика macOS: своп



Возврат страницы с диска должен проявиться в задержке.

Практика: один Go-сервис на Windows и macOS

Go

/gc/heap/live
/gc/heap/goal
GC CPU
GC assists
goroutines

Windows

Working Set
Private Bytes
Committed Bytes
жесткие страничные исключения
активность файла подкачки

macOS

Memory Pressure
Compressed
Swap Used
vm_stat
резидентная память процесса

Одни метрики Go интерпретируются в разных моделях памяти ОС.

Практика: scavenger Go на двух ОС

Windows

выделить → доступ → убрать ссылки → GC/sweep/scavenger

ожидаемый путь ОС: MEM_DECOMMIT
наблюдаем Working Set / Commit

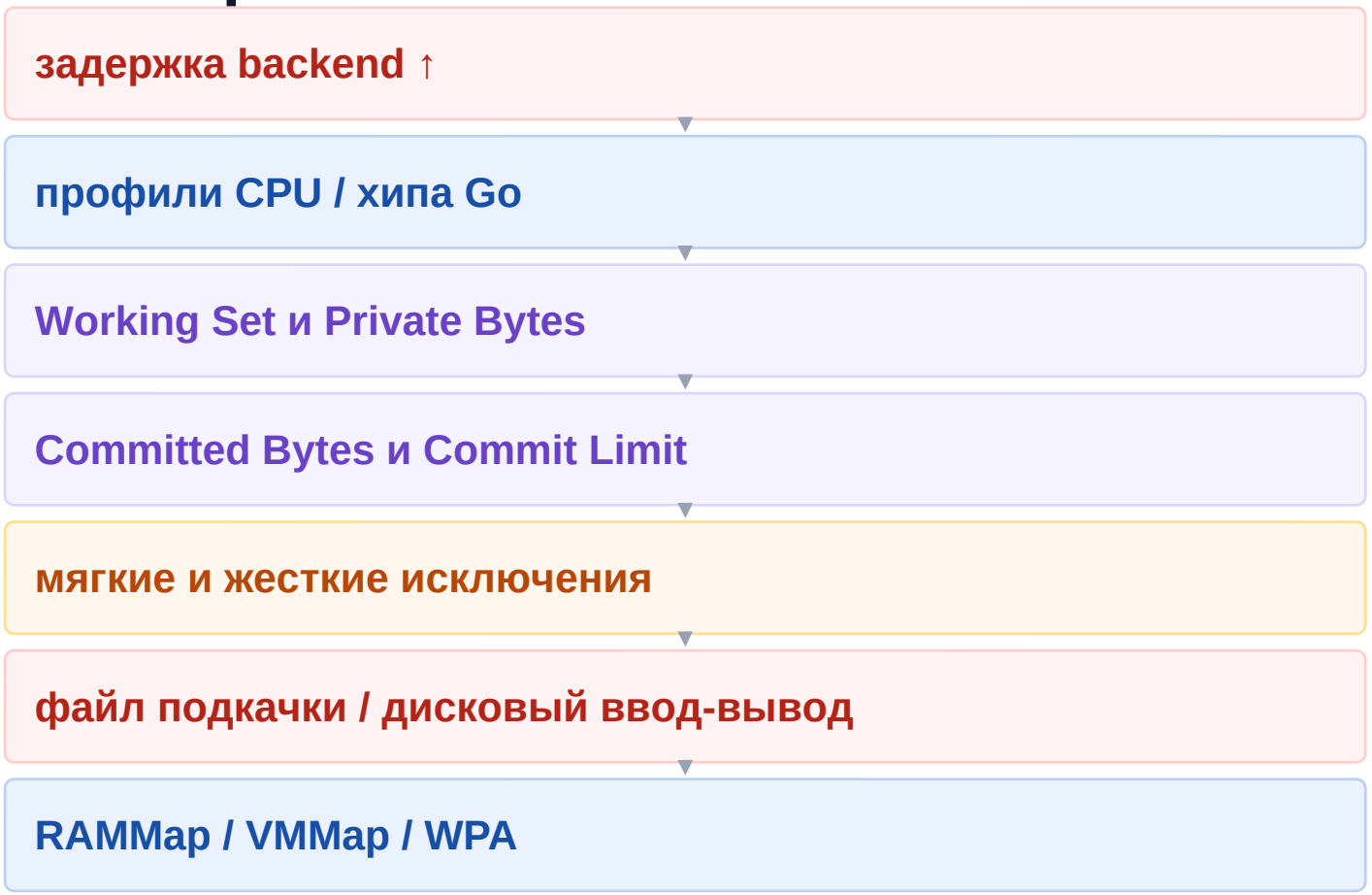
macOS

выделить → доступ → убрать ссылки → GC/sweep/scavenger

ожидаемый путь ОС: MADV_FREE_REUSABLE
наблюдаем резидентность / давление памяти

Один механизм рантайма показывает различия VM API.

Диагностическая карта Windows



Отделяем рантайм, резидентность, Commit и дисковую подкачку.

Диагностическая карта macOS



Плохие выводы Windows

Плохо

«Память в Диспетчере задач = вся память процесса»
«любое страничное исключение = чтение с диска»
«Working Set = вся память процесса»

Точнее

Разделяем Working Set, Commit/Private Bytes, типы исключений и активности подкачки.

Одной колонки «Память» недостаточно для диагноза.

Плохие выводы macOS

Плохо

«мало свободной памяти = проблема»
«Compressed > 0 = плохо»
«Swap Used > 0 = проблема»

Точнее

Смотрим Memory Pressure, сжатие / своп, исключения и задержки.

macOS активно использует RAM; важны давление и цена операций.

Сжатие не бесплатно на обеих ОС

Плюс

Уменьшает расход RAM и обращения к диску.

Цена

При повторном доступе нужна распаковка; частые переходы нагружают CPU управлением памятью.

Сжатие откладывает обращения к диску, но не устраняет дефицит RAM.

Последняя граница — накопитель

сжатия недостаточно

↓

файл подкачки / своп

↓

повторный доступ

↓

дисковый ввод-вывод

↓

задержка на порядки выше DRAM

↓

хвостовые задержки растут

Активная подкачка переносит задержку с DRAM на накопитель.

СРАВНЕНИЕ

PSI Linux и метрики настольных ОС — не одно и то же

Linux PSI

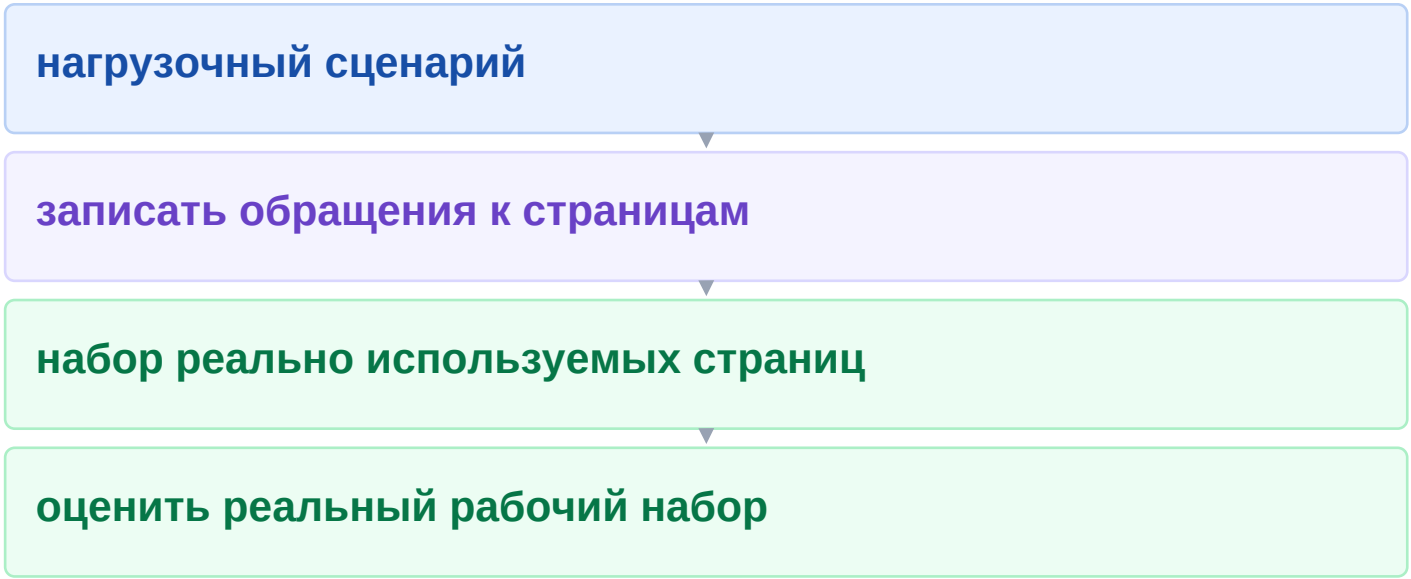
Измеряет время простоя задач из-за дефицита памяти.

Windows / macOS

Картину собираем по исключениям, подкачке, Working Set/Commit, Memory Pressure, трассировки.

Memory Pressure — сводный сигнал macOS, но это не аналог PSI 1:1.

Reference Set Windows: активные страницы сценария



Это полезно: Working Set зависит от состояния памяти ОС.

СРАВНЕНИЕ

Почему рабочий набор важнее общего размера аллокаций

Процесс A

20 ГиБ выделено, активный набор 1 ГиБ. Остальное может быть холодной / нерезидентной.

Процесс B

4 ГиБ выделено и активно. Это создает давление на физическую RAM.

Реальная цена зависит от активных страниц в RAM, не только от объема адресов.

СРАВНЕНИЕ

Живой хип Go и рабочий набор — разные виды «нужности»

живой хип Go

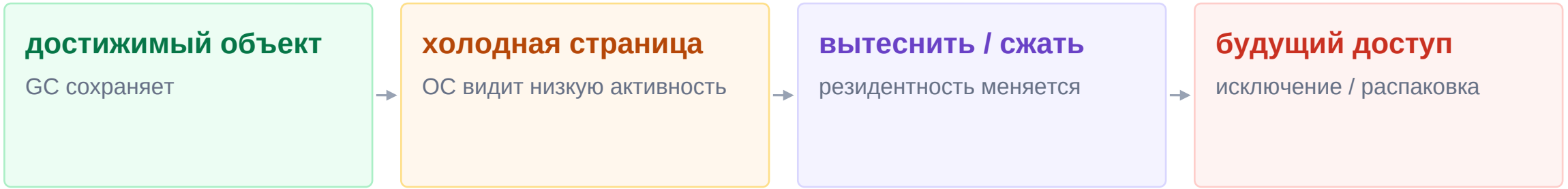
Объект достижим → GC обязан сохранить.

рабочий набор ОС

Недавно использованные страницы держим в RAM. Холодные страницы можно вытеснить.

Достижимый кеш Go может быть холодным и не занимать RAM.

Главная формула: достижимость ≠ активность



Go и диспетчер памяти ОС оценивают разные свойства данных.

Что общее, а что зависит от платформы

Устойчивая модель

виртуальное ≠ резидентное
рабочий набор
давление памяти
освобождение / вытеснение
сжатие
своп / файл подкачки
страничное исключение

Windows API

VirtualAlloc / VirtualFree
RESERVE / COMMIT
Working Set
Job Objects

macOS API

Mach VM / mmap
madvise
MADV_FREE_REUSABLE
vm_stat / vmmap

Сначала преподаем причинную модель, затем прикручиваем API и диагностику конкретной ОС.

Что зависит от реализации

- Windows: эвристики Working Set, списки страниц, сжатие и выбор кандидатов на вытеснение.
- macOS: устройство сжатия, эвристики вытеснения и свопа, очереди XNU, формула Memory Pressure.
- Внутренности полезны для углубления, но не являются постоянным контрактом.

Детали реализации отделяем от API и устойчивой модели.

Что важно запомнить

- Windows разделяет адресное пространство, Commit и Working Set.
- Reserve ≠ Commit ≠ Resident.
- Мягкое исключение обходится без диска; жесткое требует чтения.
- Файл подкачки нужен для подкачки и лимита Commit Windows.
- Сжатие на обеих ОС экономит RAM ценой CPU.
- Memory Pressure полезнее объема свободной памяти на macOS.
- Сжатая память продолжает занимать RAM в более плотном виде.
- Активная подкачка повышает p95/p99 из-за задержки накопителя.
- Достижимый объект Go может быть холодным для ОС и выйти из RAM.
- Метрики хипа Go и памяти ОС нужно анализировать вместе.

Итог: живые данные Go → виртуальная память → RAM → давление → сжатие / своп → исключение → задержка.

Финальная сравнительная модель

процесс Go

живые объекты
↓
виртуальная память

→

Windows

Reserve / Commit
↓
Working Set
↓
сокращение набора + сжатие
↓
файл подкачки
↓
жесткое исключение

→

macOS

отображение Mach VM
↓
резидентная память
↓
сжатие / освобождение
↓
своп
↓
возврат страницы

→

p99
↑

Обе ОС удерживают активные страницы в RAM; различаются API, учет и метрики.

Другой путь: когда Go больше не нужны страницы

Go

объект недостижим
↓ GC
↓ sweep
↓ scavenger

Windows

MEM_DECOMMIT

RAM можно освободить

macOS

MADV_FREE_REUSABLE

ядро может вернуть RAM

Scavenger освобождает память по решению Go; сжатие и вытеснение — решения ОС.

Основные источники лекции

- Microsoft Learn - Working Set / VirtualAlloc / Virtual Address Space and Physical Storage
- Microsoft Learn - Memory Performance Information / Cache and Memory Manager performance
- Microsoft Sysinternals - RAMMap / VMMap / Process Explorer
- Исходники рантайма Go - runtime/mem_windows.go
- Apple Support - Activity Monitor / Memory Pressure
- Apple Developer Archive - Virtual Memory Usage / Memory Allocation and VM
- Исходники рантайма Go - runtime/mem_darwin.go

Эвристики управления памятью меняются между версиями ОС; общая модель остается.